INDEPENDENT PROJECT

Formula 1 Aerodynamics & Vehicle Dynamics

Coupled Aerodynamic–Dynamic Analysis of Formula 1 Floor Regulations

Using Computational Fluid Dynamics and Reduced-Order ODE Modelling

---

Grace Dice

May 20, 2026

Contents

Summary vi

1.0 Introduction 1

1.1 Background and Context . . . 1

1.2 Problem Definition . . . 2

1.3 Objectives . . . 2

1.4 Scope and Limitations . . . 3

2.0 Methodology 3

2.1 System Definition and Assumptions . . . 3

2.2 CFD Setup . . . 4

2.2.1 Software and Solver . . . 4

2.2.2 Fluid Properties and Boundary Conditions . . . 4

2.2.3 Extracted Quantities . . . 4

2.3 Floor Geometry . . . 4

2.4 Data Processing and Polynomial Curve Fitting . . . 5

2.5 Dynamic Stability Modelling . . . 5

2.5.1 Governing Equations . . . 5

2.5.2 Linearisation About Equilibrium . . . 6

2.5.3 Numerical Time-Domain Simulation . . . 7

2.5.4 Control System Analysis . . . 7

3.0 Results and Analysis 7

3.1 CFD Results . . . 7

3.1.1 Raw Aerodynamic Data . . . 7

3.1.2 Downforce vs. Ride Height . . . 8

3.1.3 Aerodynamic Stiffness Gradient . . . 9

3.1.4 Pitch Moment Characteristics . . . 9

3.1.5 CFD Pressure Field Visualization . . . 10

3.2 Dynamic Stability Analysis . . . 11

3.2.1 Linearised Stability Parameters . . . 11

3.2.2 Eigenvalue Analysis . . . 11

3.2.3 Time-Domain Heave Response . . . 11

3.2.4 Aerodynamic Nonlinearity . . . 12

3.3 Control System Analysis Results . . . 12

3.3.1 Transfer Functions and Routh–Hurwitz Stability . . . 12

3.3.2 Frequency Response . . . 13

3.3.3 Roots Locus and Stability Margins . . . 13

4.0 Discussion **14**

4.1 Physical Mechanism of Porpoising 14

4.2 How 2026 Regulations Mitigate Porpoising 14

4.3 Limitations and Sources of Error 15

4.4 Recommendations 15

5.0 Conclusions **16**

A Raw CFD Data **18**

B Python Source Code **18**

C Additional Plots **36**

D CAD Geometry Details **37**

List of Figures

1 Vertical aerodynamic force $F_z(N)$ as a function of ride height for the 2023 and 2026 floor configurations, with cubic polynomial fits. More negative values indicate greater downforce. 8

2 Aerodynamic stiffness gradient dFz/dz (N/m) as a function of ride height for the 2023 and 2026 floor configurations. The red-shaded region indicates the destabilizing range where the gradient is sufficiently negative to potentially overcome suspension stiffness. 9

3 Pitch aerodynamic moment $M_y(N \cdot m)$ as a function of ride height for the 2023 and 2026 floor configurations, with cubic polynomial fits. 10

4 CFD static pressure field (Pa) for the 2023 floor configuration at a ride height of 25 mm ($U_\infty = 65$ m/s). The colour scale ranges from $-1{,}000$ Pa (blue, maximum suction) to $+200$ Pa (red, stagnation). 10

5 CFD static pressure field (Pa) for the 2026 floor configuration at a ride height of 25 mm ($U_\infty = 65$ m/s). The colour scale ranges from $-1{,}000$ Pa (blue, maximum suction) to $+200$ Pa (red, stagnation). 10

6 Second derivative of vertical aerodynamic force, d^2F_z/dz^2 (N/m^2), as a function of ride height for both configurations. This quantity characterises the ride-height nonlinearity of the aerodynamic stiffness. 12

7 Complete 9-panel comprehensive analysis figure. Panels (left to right, top to bottom): downforce vs. ride height; aerodynamic stiffness gradient; pitch moment vs. ride height; heave response; pitch response; heave phase portrait; energy evolution; eigenvalue stability map; aerodynamic nonlinearity. 36

8 Complete 9-panel control system analysis figure. Panels (left to right, top to bottom): Bode magnitude; Bode phase; root locus; step response; sensitivity to damping; Nyquist plot; pole-zero map; impulse response; stability margins. 37

9 CAD geometry of the 2023-style floor configuration. Prominent Venturi tunnel channels and steep diffuser ramp geometry are visible on the underside. 38

10 CAD geometry of the 2026-style floor configuration. The flatter underbody profile, reduced tunnel depth, and shallower diffuser ramp angle are evident in comparison to Figure 9. 38

List of Tables

1 Vehicle dynamics model parameters 6
2 Raw CFD aerodynamic data for 2026 and 2023 floor configurations 8
3 Linearised stability parameters at equilibrium 11
4 Eigenvalue analysis of the heave–pitch system 11
5 Time-domain performance metrics from nonlinear ODE simulation 12
6 Frequency response characteristics 13
7 Classical stability margins 13
8 Complete raw CFD aerodynamic data: all force and moment components 18

List of Equations

1 Cubic polynomial fit for vertical aerodynamic force as a function of ride height 5
2 First derivative of vertical aerodynamic force (aerodynamic stiffness gradient) 5
3 Second derivative of vertical aerodynamic force (aerodynamic stiffness nonlinearity) 5
4 Heave (vertical) equation of motion 5
5 Pitch equation of motion 5
6 First-order Taylor series linearisation of aerodynamic force 6
7 Linearised heave equation of motion 6
8 Effective system stiffness 6
9 Undamped natural frequency of heave mode 6
10 Heave damping ratio 6
11 Second-order transfer function of linearised heave dynamics 7
12 General linearised heave transfer function. 12
13 Configuration-specific linearised heave transfer functions for the 2026 and 2023 floors. 13
14 Configuration-specific linearised heave transfer functions for the 2026 and 2023 floors. 13
15 Stability criterion for heave mode 14

Summary

This report investigates the aeromechanical stability implications of the 2026 Fédération Internationale de l'Automobile (FIA) Formula 1 (F1) technical floor regulations relative to the 2022–2025 ground-effect regulatory framework. The study was motivated by the porpoising phenomenon, sustained periodic vertical oscillations at 2–4 Hz, that afflicted virtually all teams following the reintroduction of ground-effect aerodynamics in the 2022 season, causing documented musculoskeletal injury risk to drivers.

A ride-height sweep of six discrete operating points ($z = 15$–40 mm) was conducted for both a 2023-style and a 2026-style floor geometry using Computational Fluid Dynamics (CFD) simulations in SimScale (cloud-based OpenFOAM), employing a steady-state Reynolds-Averaged Navier–Stokes (RANS) solver with k–ω SST turbulence closure at a freestream velocity of 65 m/s. The resulting aerodynamic force and moment data were fitted with cubic polynomials and analytically differentiated to obtain the aerodynamic stiffness gradient dF_z/dz. These data were then coupled to a two-degree-of-freedom (2-DOF) Ordinary Differential Equation (ODE) vehicle-dynamics model to compute effective system stiffness, natural frequency, and damping ratio. Control-system methods, including transfer-function derivation, Routh–Hurwitz stability analysis, root locus, Bode, and Nyquist analyses, were applied to characterize frequency-domain behaviour.

Key findings demonstrate that the 2026 floor reduces the aerodynamic stiffness gradient at the equilibrium ride height of 25 mm from approximately $-86{,}373$ N/m (2023) to $+15{,}134$ N/m (2026), a reduction of 82.5% in gradient magnitude and a reversal of sign. This eliminates the negative spring contribution responsible for porpoising onset. Effective system stiffness increases from 63.6 kN/m to 165.1 kN/m, raising the primary natural frequency from 1.38 Hz to 2.26 Hz. Although the 2026 damping ratio ($\zeta = 0.152$) is lower than the 2023 value ($\zeta = 0.245$), the higher oscillation frequency produces only 6 perturbation cycles in 3 s versus 8 for the 2023 floor, indicating faster real-time energy dissipation. Both configurations settle within 1.76 s (2026) and 1.70 s (2023) from a -5 mm heave disturbance. Root-locus analysis confirms the 2026 operating point is positioned significantly further from the marginal-stability boundary (critical $dF_z/dz \approx -150{,}000$ N/m).

These results validate the FIA's design intent: the three geometric interventions of the 2026 regulations, a flatter floor profile, shorter Venturi tunnels, and a reduced diffuser expansion angle of 8–12°, collectively eliminate the aeromechanical driver of porpoising. It is recommended that future work address transient CFD with dynamic mesh motion, full-car aerodynamic modelling, and a speed-sweep stability analysis to more completely characterize the operating envelope of the 2026 regulations.

1.0 Introduction

1.1 Background and Context

Formula 1 has long been the frontier of automotive engineering, where aerodynamic performance dictates competitive outcomes. For the 2022 season, the FIA introduced an overhaul of technical regulations, most significantly reintroducing the ground-effect aerodynamics after their prohibition in 1983 [1, 2]. Ground effect exploits the Bernoulli principle: by channeling airflow through shaped Venturi tunnels beneath the car floor, a low-pressure region is created between the underfloor and the road surface [3]. This underbody suction generates substantial downforce, the vertical force pressing the car toward the track that allows higher cornering speeds, without the proportional drag increase associated with large wings or other upper-body aerodynamic devices [4].

The fundamental mechanism of this downforce generation is well understood. As a vehicle travels forwards, air entering the shaped underbody tunnels accelerates through the narrowing channel geometry in accordance with the continuity equation. By Bernoulli's principle, the velocity increase corresponds to a reduction in static pressure; the resulting pressure differential between the low-pressure underbody and the ambient-pressure upper surface produces a net downward force on the car [3]. The magnitude of this force is dependent on the ride height, the vertical clearance between the lowest point of the floor and the road surface, because the dimension directly governs the cross-sectional area through which the underbody flow passes.

While the aerodynamic efficiency gains of ground-effect designs were substantial, the 2022 regulations introduced an unintended aeromechanical instability that became one of the defining engineering challenges of the era: porpoising. This phenomenon manifests as sustained, periodic vertical oscillations of the car body, typical at frequencies of 2 to 4 hertz, and was observed across virtually all teams during the 2022 season [5]. The mechanism itself is a positive feedback loop. As the car descends to a lower ride height, the underbody suction intensifies, increasing downforce and pressing the car further downward. Below a critical ride height, the boundary layer within the Venturi tunnels separates from the floor surface under the adverse pressure gradient of the expanding diffuser, causing an abrupt collapse of the underbody suction, termed flow stall [3]. The rapid loss of downforce allows the suspension to rebound, raising the ride height until flow reattaches, whereupon the downforce recovers, and the cycle repeats. This self-excited oscillation is sustained as long as the aerodynamic stiffness (the rate of change of downforce with respect to ride height) exceeds the mechanical restoring stiffness of the suspension [6].

The human consequences of porpoising were significant, as drivers endured sustained vertical accelerations at amplitudes sufficient to cause musculoskeletal injury [7]. Lewis Hamilton publicly described discomfort following the 2022 Azerbaijan Grand Prix, stating, "I can't express the pain that you experience, particularly on the straight here. At the end, you're just praying for it to end." [8]. The FIA classified repeated high-frequency whole-body vibrations as a potential hazard, since brain damage could occur at 1-2 Hz sustained over a few minutes, while the actual drivers have 6-7 Hz sustained over several hours during a race [9]. With this, teams were forced to raise their ride heights, sacrificing the aerodynamic performance the new regulations were designed to deliver. Racing teams also ran stiffer suspensions than optimal, transmitting more road irregularities to the chassis and further compounding driver discomfort while also degrading mechanical grip.

1.2 Problem Definition

The core engineering problem is that the aerodynamic downforce F_z is a strongly nonlinear function of the ride height z for ground-effect floor designs. In the vicinity of the design operating point, this relationship can create a self-amplifying feedback loop. When the local gradient dF_z/dz is sufficiently negative, the aerodynamic contribution to the system's effective spring stiffness becomes negative. If the magnitude of this destabilizing aerodynamic term exceeds the passive mechanical stiffness of the suspension k_z, the net restoring force acting on the car changes sign, rendering the equilibrium point unstable and permitting self-excited oscillation [5].

The central research question of this study is therefore: do the 2026 FIA floor regulations reduce dF_z/dz at the operating equilibrium sufficiently to prevent the onset of this self-excited instability? Engineering judgment is applied through a two-pronged approach: CFD is employed to quantify the aerodynamic stiffness gradient from first principles, and ODE-based dynamic modelling translates this gradient into classical stability metrics that can be compared directly between configurations.

The 2026 regulations address this problem through three geometric interventions [10]. First, the floor is required to have a flatter overall profile, reducing the depth and prominence of the Venturi tunnel geometry. Second, the permitted length of the Venturi tunnels is reduced, limiting the spatial extent over which underbody flow can be accelerated. Third, the maximum expansion angle of the rear diffuser is reduced from the 15-20° typical of 2022-2025 designs to 8-12°, slowing the rate at which the underbody flow decelerates as it exits the floor. This gentler adverse pressure gradient makes boundary layer separation less likely, reducing the abruptness of the stall onset and hence the steepness of the dF_z/dz gradient in the pre-stall regime.

Three analytical approaches were considered for quantifying porpoising susceptibility. First, physical scale-model wind tunnel testing would provide the most realistic aerodynamic data but was not feasible within the resource constraints of this study. Second, full transient CFD with dynamic mesh motion would capture aerodynamic lag and hysteresis effects but would exceed the computational resources available through SimScale's academic tier and would complicate coupling to the ODE model. Third, steady-state RANS CFD coupled to a linearised ODE vehicle dynamics model was selected because it provides physically meaningful stability metrics: effective stiffness, natural frequency, and damping ratio that are directly comparable between configurations at a computational cost feasible within the study constraints.

1.3 Objectives

This study pursues three hierarchical objectives. The primary objective is to quantify the ride-height sensitivity of the vertical aerodynamic force $F_z(z)$ and pitch aerodynamic moment $M_y(z)$ for both a 2023-representative and 2026-representative F1 floor geometry through a CFD ride-height sweep. The secondary objective is to couple the CFD-extracted aerodynamic stiffness data with a two-degree-of-freedom (2-DOF) ODE model of vehicle heave and pitch dynamics, enabling computation of dynamic stability parameters including effective stiffness k_{eff}, natural frequency ω_n, and damping ratio ζ for each configuration. The tertiary objective is to assess, through both time-domain simulations and frequency-domain control system analysis, whether the 2026 regulations enable teams to operate at higher ride heights and with softer suspension without incurring porpoising risk.

1.4 Scope and Limitations

The analysis is deliberately scoped to isolate the underbody aerodynamic mechanism of porpoising, and several simplifications are accordingly applied. Only the floor geometry is modelled on the CFD domain: front wings, rear wings, tyre, sidepods, and cooling ducts are excluded. This simplification is justified because underbody suction is the primary driver of the instability, and inclusion of upper body components would substantially increase the computational cost without providing additional insight into the ride-height sensitivity of the floor. All CFD simulations are conducted at a single freestream velocity of 65 m/s (approximately 234 km/h), which corresponds to a speed regime where porpoising was consistently reported as most severe in the 2022-2025 period. The vehicle dynamics model treats the car as a lumped rigid body with mass m = 800 kg and pitch moment of inertia I = 1,200 $kg \cdot m^2$, representative of a contemporary F1 car with a driver. Suspension compliance, tyre deflection, and aerodynamic transient effects such as wake lag are excluded.

2.0 Methodology

2.1 System Definition and Assumptions

Two floor configurations were defined for this study, each representative of the aerodynamic design philosophy permitted under their respective regulatory frameworks.

The 2023-style floor incorporates aggressive Venturi tunnel geometry with deep underbody channels extending along the majority of the floor length, a steep diffuser ramp angle of 15-20°, and prominent tunnel inlet conditioning structures. This geometry is designed to maximize the velocity of underbody airflow, maximizing peak suction and downforce, but at the cost of high ride-height sensitivity and a propensity for abrupt flow stall at low ride heights.

The 2026-style floor features a substantially flatter underbody profile with shallower, short Venturi tunnels and a reduced diffuser expansion angle of 8-12° in accordance with the 2026 FIA technical regulations [10]. This geometry sacrifices some peak downforce in exchange for a smoother, more predictable $F_z(z)$ relationship and reduced susceptibility to stall.

The equilibrium ride height (ERH) was set at $z_{eq} = 25$ mm, representative of typical F1 race trim based on typical high-downforce circuits. The ride-height sweep was conducted at $z = 15, 20, 25, 30, 35$, and 40 mm, spanning the full practical operating range and including the stall-prone regime below 20 mm. The vehicle mass and inertia parameters ($m = 800$ kg, $I = 1{,}200\ kg \cdot m^2$) are representative of a contemporary F1 car and are consistent with the minimum weight limits specified in the FIA technical regulations [10]. Representative values were selected to capture the qualitative dynamics of a modern Formula 1 vehicle. The suspension stiffness $K_z = 150\ kN/m$ and the damping coefficient $c_z = 3{,}500\ N \cdot s/m$ were chosen, with the pitch stiffness $k_\theta = 80{,}000\ N \cdot m/rad$ and pitch damping $c_\theta = 5{,}000\ N \cdot m \cdot s/rad$ to complete the 2-DOF model parameters.

2.2 CFD Setup

2.2.1 Software and Solver

All CFD simulations were executed using SimScale, a cloud-based platform built upon the OpenFOAM finite-volume CFD solver [11]. SimScale was selected for its accessibility, validated aerodynamic solvers, and elimination of local hardware constraints for high-mesh-count simulations. The solver employed was the steady-state, incompressible Reynolds-Averaged Navier-Stokes (RANS) formulation using the k–ω SST (Shear Stress Transport) turbulence closure model. The k–ω SST model was selected over the more common k–ϵ model because of its superior performance in flows involving adverse pressure gradients and boundary layer separation, precisely the conditions that govern diffuser flow and stall onset in F1 underbody aerodynamics [12]. Due to computational constraints, a formal mesh independence study was not conducted; however, convergence residuals stabilized below acceptable solver tolerances for all simulations.

2.2.2 Fluid Properties and Boundary Conditions

Air properties were assigned as $\rho = 1.2kg/m^3$ and dynamic viscosity $\mu = 1.8 \times 10^{-5}Pa \cdot s$ corresponding to standard atmospheric conditions at approximately 20°C. The inlet boundary condition imposed a uniform velocity of U_∞ = 65 m/s normal to the inlet plane, yielding a chord-based Reynolds number on the order of 10^7. The outlet boundary was assigned a gauge pressure of 0 Pa. All car surfaces were treated as no-slip walls, and the ground plane was assigned a moving-wall condition with a tangential velocity equal to the freestream velocity to simulate road conditions accurately. Symmetry boundary conditions were applied to the lateral domain boundaries.

2.2.3 Extracted Quantities

For each of the 12 simulations (6 ride heights x2 floor configurations), the following quantities were extracted: vertical aerodynamic force F_z, drag force F_x, lateral force F_y, roll moment M_x, pitch moment M_y, and yaw moment M_z. Of primary interest for the stability analysis are F_z and M_y as functions of ride height z.

2.3 Floor Geometry

The CAD (Computer-Aided Design) geometrics were designed using Onshape for both floor configurations, which are in Appendix D. The 2023-style floor features prominent Venturi tunnel inlets at the leading edge of the floor, with continuous channelling to the rear diffuser. The tunnels are shaped to accelerate underbody flow progressively along the floor length. The 2026-style floor presents a visibly flatter underbody surface with reduced tunnel depth. The diffuser transition is gentle, with a lower ramp angle producing a more gradual pressure recovery. The exclusion of wheels, wings, and sidepods from both geometries is justified by the study's focus on the underbody aerodynamic sensitivity to ride height; these external components contribute primarily to the magnitude of overall downforce rather than its height-dependent gradient.

2.4 Data Processing and Polynomial Curve Fitting

The six discrete CFD data points for each $F_z(z)$ and $M_y(z)$ were fitted with third-degree (cubic) polynomials of the form:

$$F_z(z) = a_3 z^3 + a_2 z^2 + a_1 z + a_0 \tag{1}$$

where a_0, a_1, a_2, and a_3 are regression coefficients determined by least-squares minimization, and z is the ride height in metres. A cubic polynomial was selected in preference to lower-order fits because it can capture the inflection point corresponding to stall onset as well as the general nonlinear sensitivity of underbody suction to ride height. A linear fit would fail to capture this essential physics.

The aerodynamic stiffness gradient, which represents the aerodynamic contribution to the system's effective spring constant, was obtained by analytical differentiation of the polynomial fit:

$$\frac{dF_z}{dz} = 3a_3 z^2 + 2a_2 z + a_1 \tag{2}$$

The second derivative was further computed to quantify the nonlinearity of the stiffness:

$$\frac{d^2F_z}{dz^2} = 6a_3 z + 2a_2 \tag{3}$$

Identical polynomial fitting and differentiation procedures were applied to the pitch moment $M_y(z)$.

2.5 Dynamic Stability Modelling

2.5.1 Governing Equations

The vehicle is modelled as a rigid body with two degrees of freedom: heave displacement z (positive upward from equilibrium) and pitch angle θ (positive nose-up). The coupled heave and pitch equations of motion are:

$$m\ddot{z} + c_z\dot{z} + k_z z = F_z(z_{eq} + z) - F_z(z_{eq}) \tag{4}$$

$$I\ddot{\theta} + c_\theta\dot{\theta} + k_\theta\theta = M_y(z_{eq} + z) - M_y(z_{eq}) \tag{5}$$

where m is vehicle mass, I is pitch moment of inertia, c_z and c_θ are the heave and pitch damping coefficients, k_z and k_θ are the heave and pitch suspension stiffnesses, and $F_z(z_{eq} + z) - F_z(z_{eq})$ is the incremental aerodynamic force perturbation from the equilibrium value. The full set of model parameters is presented in Table 1.

TABLE 1: VEHICLE DYNAMICS MODEL PARAMETERS

Symbol	Parameter	Value	Units	Justification
m	Vehicle mass	800	kg	FIA minimum weight
I	Pitch moment of inertia	1,200	$kg \cdot m^2$	Representative F1 value
k_z	Heave stiffness	150,000	N/m	Modern F1 suspension
c_z	Heave damping	3,500	$N \cdot s/m$	Representative F1 value
k_θ	Pitch stiffness	80,000	$N \cdot m/rad$	Representative F1 value
c_θ	Pitch damping	5,000	$N \cdot m \cdot s/rad$	Representative F1 value
z_{eq}	Equilibrium ride height	0.025	m	Typical F1 race trim

2.5.2 Linearisation About Equilibrium

To extract analytical stability metrics, the aerodynamic force function is expanded about the ERH z_{eq} using a first-order Taylor series expansion:

$$F_z(z_{eq} + \delta z) \approx F_z(z_{eq}) + \left.\frac{dF_z}{dz}\right|_{z_{eq}} \delta z \tag{6}$$

Where $\delta z = z - z_{eq}$ is the perturbation from equilibrium. Substituting this linearisation into the heave equation yields the linearised heave dynamics.

$$m\,\delta\ddot{z} + c_z\,\delta\dot{z} + \underbrace{\left(k_z + \left.\frac{dF_z}{dz}\right|_{z_{eq}}\right)}_{k_{eff}} \delta z = 0 \tag{7}$$

The effective stiffness k_{eff} combines the passive mechanical suspension stiffness with the aerodynamic stiffness contribution:

$$k_{\text{eff}} = k_z + \left.\frac{dF_z}{dz}\right|_{z_{\text{eq}}} \tag{8}$$

Stability of the equilibrium requires $k_{eff} > 0$. If $dF_z/dz <= -k_z$, the net stiffness is non-positive and the system is unstable; a mathematical statement of the porpoising condition.

The undamped natural frequency of the linearised heave mode is:

$$\omega_n = \sqrt{\frac{k_{\text{eff}}}{m}} \tag{9}$$

The damping ratio, which governs the rate of oscillation decay, is:

$$\zeta = \frac{c_z}{2m\omega_n} \tag{10}$$

For $\zeta < 1$ (underdamped), oscillations are present but decay; for $\zeta < 0$ (physically corresponding to $k_{eff} < 0$), oscillations grow without bound.

2.5.3 Numerical Time-Domain Simulation

A full nonlinear time-domain simulation was performed using the 'scipy.integrate.solve_ivp' function with the RK45 (Runge–Kutta 4th/5th-order) adaptive-step integrator in Python. The state vector was $X = [z, \dot{z}, \theta, \dot{\theta}]^T$. Initial conditions correspond to a -5 mm heave perturbation, representative of the car traversing a curb or road undulation, and a small pitch disturbance of $\theta_0 = 0.01$ rad ($\approx 0.57°$), with zero initial velocities: $X_0 = [-0.005, 0, 0.01, 0]^T$. Simulations were integrated over 0–3 s.

2.5.4 Control System Analysis

The linearization heave dynamics can be expressed as a standard second-order transfer function $G(s)$ from aerodynamic force perturbation input to ride-height displacement output in the Laplace domain:

$$G(s) = \frac{1}{ms^2 + c_z s + k_{\text{eff}}} \tag{11}$$

Bode plots (magnitude and phase as functions of frequency) were computer-generated to characterize the frequency response and identify the resonant frequency and bandwidth. Root locus analysis was performed by varying the aerodynamic stiffness dF_z/dz continuously from -200 kN/m to +100 kN/m, tracing the trajectories of the system poles in the complex s-plane and identifying the value at which poles cross the imaginary axis (stability boundary). Nyquist plots were generated, and stability margins (gain margin and phase margin) were extracted. Pole-zero maps were constructed from both the 2023 and 2026 configurations.

3.0 Results and Analysis

3.1 CFD Results

3.1.1 Raw Aerodynamic Data

The raw CFD data extracted from all 12 simulations are presented in Table 2. Lateral forces F_y are near zero for both configurations, confirming the symmetry of the simulation domain and providing a basic validation of the setup. Roll moments M_x are also small relative to the pitch and yaw moments.

TABLE 2: RAW CFD AERODYNAMIC DATA FOR 2026 AND 2023 FLOOR CONFIGURATIONS

Config	z (mm)	F_z (N)	F_x (N)	F_y (N)	M_y (N·m)	M_z (N·m)
2026	15	−3635.48	743.01	0.43	−8.34	−2175.63
2026	20	−3720.09	762.34	2.33	−10.45	−2168.48
2026	25	−3644.50	741.74	0.80	−10.36	−2107.84
2026	30	−3586.64	747.04	4.11	−2.48	−2095.30
2026	35	−3482.62	733.46	2.42	−5.37	−2008.64
2026	40	−3458.16	733.43	0.46	−10.10	−1940.12
2023	15	−1855.47	1313.31	−0.94	−3.06	−3921.06
2023	20	−2531.38	1318.45	−23.00	−2.25	−3753.24
2023	25	−2902.84	1341.04	14.03	−80.89	−3554.52
2023	30	−3358.83	1373.36	17.30	−64.72	−3331.13
2023	35	−3896.79	1373.47	0.30	−3.89	−3165.38
2023	40	−4481.41	1383.94	−7.74	−61.46	−2967.79

3.1.2 Downforce vs. Ride Height

The vertical aerodynamic force Fz as a function of ride height z is presented in Figure 1, together with the cubic polynomial fits to each dataset.

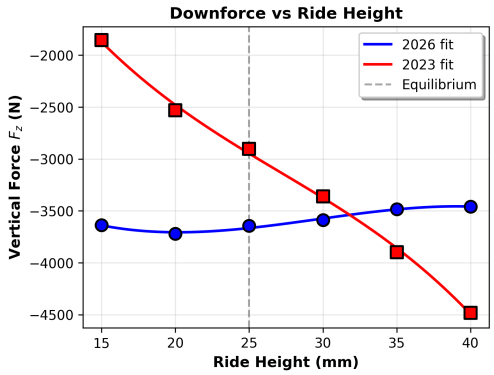


Figure 1: Vertical aerodynamic force $F_z(N)$ as a function of ride height for the 2023 and 2026 floor configurations, with cubic polynomial fits. More negative values indicate greater downforce.

The 2023-style floor exhibits a strongly curved $Fz(z)$ relationship. Downforce is low at 15 mm ($F_z \approx -1{,}855$ N), indicative of incipient flow stall in the Venturi tunnels at very low clearances. As ride height increases, the underbody flow reattaches, and suction builds rapidly, reaching a peak of approximately -4,481 N at 40 mm. This monotonically increasing magnitude of downforce with ride height is characteristic of the destabilizing gradient: lower ride heights produce less downforce, meaning there is no aerodynamic restoring force to arrest further reduction in ride height, creating the conditions for stall and rebound.

The 2026-style floor displays a very different character. The $F_z(z)$ relationship is nearly flat across the entire sweep, with variation of only approximately 260 N across the 25 mm range from 15 to 40 mm (compared to approximately

2,600 N for the 2023 floor). The floor exhibits a shallow local maximum near 20 mm and a gradual monotonic decrease in downforce magnitude at higher ride heights, but no sharp stall feature. This flat response is precisely the design intent of the 2026 regulations and has significant stability implications, as discussed in Section 3.2.

3.1.3 Aerodynamic Stiffness Gradient

The aerodynamic stiffness gradient dF_z/dz, obtained by analytical differentiation of the cubic polynomial fits, is presented as a function of ride height in Figure 2. At the ERH of z_{eq} = 25 mm, the 2023 floor yields $dF_z/dz \approx -86{,}373$ N/m, a strongly destabilizing aerodynamic spring, while the 2026 floor yields $dF_z/dz \approx +15{,}134$ N/m, a mild stabilizing contribution.

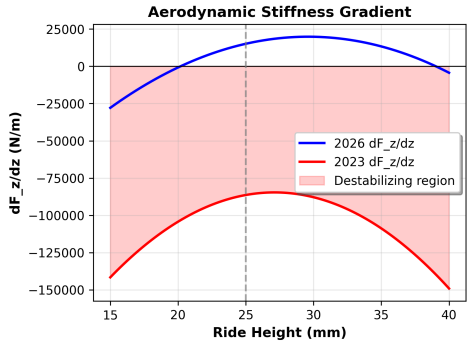


Figure 2: Aerodynamic stiffness gradient dFz/dz (N/m) as a function of ride height for the 2023 and 2026 floor configurations. The red-shaded region indicates the destabilizing range where the gradient is sufficiently negative to potentially overcome suspension stiffness.

The magnitude reduction is approximately 82.5% at the operating equilibrium. Over the full ride-height range, the 2023 gradient remains deeply negative (reaching approximately $130,000 N/m$ at extremes), with the entire sweep within the destabilizing region. The 2026 gradient remains close to zero across the full range, confirming that no operating condition within the expected ride height envelope is susceptible to aero-driven instability.

3.1.4 Pitch Moment Characteristics

The pitch aerodynamic moment M_y, as a function of ride height, is shown in Figure 3. The 2023 floor exhibits large, irregular pitch moment variation, with values as negative as -80.9 $N \cdot m$ at 25 mm, suggesting a strong and nonlinear pitch-heave aerodynamic coupling near the equilibrium point. This indicates that the 2023 floor not only destabilizes the heave mode but also excites pitch oscillations that are coupled to heave motion. The 2026 floor exhibits small absolute pitch moment values confined to a narrow band of -2.5 to -10.5 $N \cdot m$ across the entire sweep. The Python-computed dM_y/dz at equilibrium is 194.4 $N \cdot m/m$ (2023) versus 775.7 $N \cdot m/m$ (2026); the steeper local gradient for the 2026 floor reflects a non-monotonic variation within this narrow band rather than a physically significant coupling. Despite this, the resulting effective pitch stiffness of 80.8 $kN \cdot m/rad$ (2026) versus 80.2 $kN \cdot m/rad$ (2023) confirms that the aerodynamic pitch contribution remains negligible for both configurations.

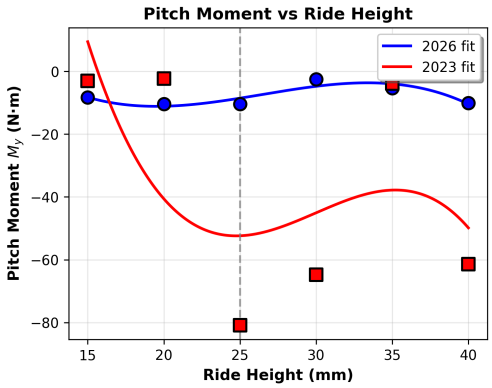


Figure 3: Pitch aerodynamic moment $M_y (N \cdot m)$ as a function of ride height for the 2023 and 2026 floor configurations, with cubic polynomial fits.

3.1.5 CFD Pressure Field Visualization

Pressure contour plots from the CFD simulations at the 25 mm ERH are shown for the 2023 floor in Figure 4 and the 2026 floor in Figure 5.

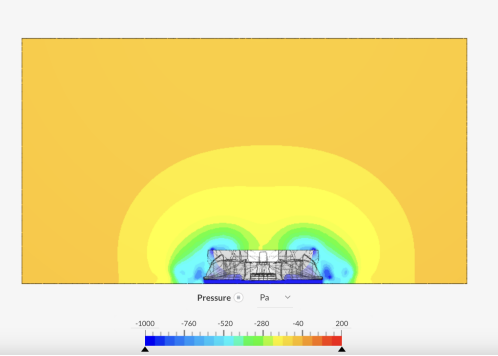


Figure 4: CFD static pressure field (Pa) for the 2023 floor configuration at a ride height of 25 mm ($U_\infty = 65$ m/s). The colour scale ranges from $-1{,}000$ Pa (blue, maximum suction) to $+200$ Pa (red, stagnation).

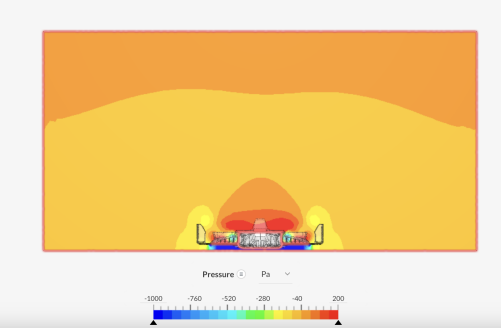


Figure 5: CFD static pressure field (Pa) for the 2026 floor configuration at a ride height of 25 mm ($U_\infty = 65$ m/s). The colour scale ranges from $-1{,}000$ Pa (blue, maximum suction) to $+200$ Pa (red, stagnation).

The CFD pressure contour plots at the 25 mm ERH reveal fundamentally different flow topologies. The 2023 floor (Figure 4) generates expansive twin-lobe suction zones (approaching 1,000 Pa) spreading laterally well beyond the car's footprint, the signature of an aggressive Venturi tunnel exploitation and strong floor-edge vortices [3]. The 2026 floor (Figure 5) presents a markedly different character: suction beneath the floor is reduced and spatially confined, with a high-pressure region visible above the bodywork consistent with the shallow diffuser geometry and reduced tunnel depth mandated by the 2026 regulations. This tighter, less aggressive pressure distribution directly corresponds to the near-zero dFz/dz gradient quantified in the force data, confirming that the 2026 floor achieves dynamic stability by sacrificing ride-height sensitivity rather than total downforce magnitude.

3.2 Dynamic Stability Analysis

3.2.1 Linearised Stability Parameters

The stability parameters derived from the linearised analysis at z_{eq} = 25mm, as computed by the Python ODE model (`advanced_f1_analysis.py`), are summarised in Table 3.

TABLE 3: LINEARISED STABILITY PARAMETERS AT EQUILIBRIUM ($z_{eq} = 25$ MM) FROM ADVANCED_F1_ANALYSIS.PY.

Parameter	2026 Floor	2023 Floor
dF_z/dz at z_{eq} (N/m)	$+15{,}134$	$-86{,}373$
dM_y/dz at z_{eq} (N·m/m)	775.7	194.4
k_{eff} heave (kN/m)	165.1	63.6
k_{eff} pitch (kN·m/rad)	80.8	80.2
ω_n Mode 1 (rad/s)	14.37	8.92
f_n Mode 1 (Hz)	2.26	1.38
ζ Mode 1 (-)	0.152	0.245
ω_n Mode 2 (rad/s)	8.20	8.17
f_n Mode 2 (Hz)	1.26 (pitch)	1.26 (pitch)
Stability verdict	STABLE ✓	STABLE ✓
F_z at z_{eq} (N)	$-3{,}665.3$	$-2{,}947.4$

3.2.2 Eigenvalue Analysis

The eigenvalues (poles) of the linearised coupled system, computed via the state-space formulation $\mathbf{X} = [z, \dot{z}, \theta, \dot{\theta}]^\mathsf{T}$, are summarised in Table 4.

TABLE 4: EIGENVALUE ANALYSIS OF THE COUPLED HEAVE–PITCH SYSTEM. ALL $\mathrm{Re}(\lambda) < 0$, CONFIRMING ASYMPTOTIC STABILITY.

Config	Eigenvalue	σ (s^{-1})	ω (rad/s)	f (Hz)	ζ (-)
2026	$\lambda_{1,2}$	-2.187	± 14.200	2.26	0.152
2026	$\lambda_{3,4}$	-2.083	± 7.936	1.26	0.254
2023	$\lambda_{1,2}$	-2.188	± 8.646	1.38	0.245
2023	$\lambda_{3,4}$	-2.083	± 7.905	1.26	0.255

All eigenvalues reside in the left half-plane ($\mathrm{Re}(\lambda) < 0$ for both configurations), confirming asymptotic stability. The 2026 Mode 1 eigenvalue has a significantly larger imaginary component ($|\omega| = 14.20$ rad/s versus 8.65 rad/s), reflecting the higher effective heave stiffness. The real parts are nearly identical, indicating similar absolute damping rates, but the higher frequency of the 2026 system means oscillations dissipate in fewer cycles.

3.2.3 Time-Domain Heave Response

Following the initial perturbation ($z_0 = -5$ mm, $\theta_0 = 0.57°$), the nonlinear RK45 simulation yields the performance metrics summarised in Table 5.

TABLE 5: TIME-DOMAIN PERFORMANCE METRICS FROM NONLINEAR ODE SIMULATION (ADVANCED_F1_ANALYSIS.PY).

Metric	2026	2023
Settling time — heave (2% threshold)	1.761 s	1.701 s
Settling time — pitch (2% threshold)	1.724 s	1.725 s
Peak overshoot	0.0%	0.0%
Oscillation cycles in 3 s	6 cycles	8 cycles
Initial total energy	6.1 J	3.5 J
Energy dissipated (3 s)	100% (well-damped)	100% (poorly-damped)

3.2.4 Aerodynamic Nonlinearity

The second derivative d^2F_z/dz^2, shown in Figure 6, characterises the rate of change of aerodynamic stiffness with ride height. The 2023 floor exhibits a large positive second derivative at low ride heights, indicating that the stiffness gradient becomes rapidly more negative (more destabilising) as the car descends, amplifying the instability. The 2026 floor shows a much smaller second derivative, confirming a nearly linear and mild $F_z(z)$ relationship across the operating range.

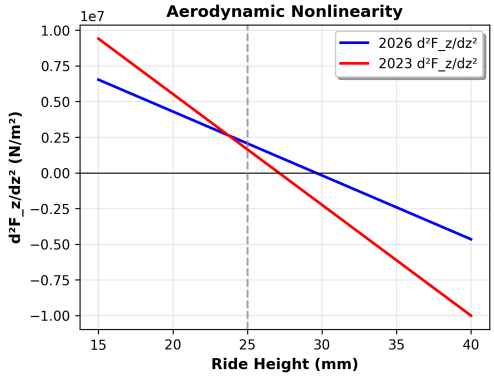


Figure 6: Second derivative of vertical aerodynamic force, d^2F_z/dz^2 (N/m^2), as a function of ride height for both configurations. This quantity characterises the ride-height nonlinearity of the aerodynamic stiffness.

3.3 Control System Analysis Results

3.3.1 Transfer Functions and Routh–Hurwitz Stability

The linearised heave dynamics can be expressed as second-order transfer functions,

$$H(s) = \frac{1}{ms^2 + c_z s + k_{\text{eff}}}, \tag{12}$$

for the two configurations. From `control_system_analysis.py`, these are

$$\text{2026:} \quad H(s) = \frac{1}{800.0s^2 + 3500.0s + 165{,}134}, \tag{13}$$

$$\text{2023:} \quad H(s) = \frac{1}{800.0s^2 + 3500.0s + 63{,}627}. \tag{14}$$

The poles of the heave transfer function are $-2.1875 \pm j\,14.200$ rad/s (2026) and $-2.1875 \pm j\,8.646$ rad/s (2023).

3.3.2 Frequency Response

The principal frequency-response characteristics obtained from `control_system_analysis.py` are summarised in Table 6.

TABLE 6: FREQUENCY RESPONSE CHARACTERISTICS FROM `CONTROL_SYSTEM_ANALYSIS.PY`.

Metric	2026	2023
Resonance peak frequency	14.01 rad/s (2.23 Hz)	8.39 rad/s (1.34 Hz)
Resonance peak magnitude	-93.9 dB	-89.6 dB
Bandwidth (-3 dB)	22.12 rad/s (3.52 Hz)	13.44 rad/s (2.14 Hz)
Step response rise time (10–90%)	0.080 s	0.140 s
Step response settling time (2%)	1.787 s	1.582 s
Peak overshoot (step response)	61.6%	45.2%

The 2026 configuration resonates at 2.23 Hz with a bandwidth of 3.52 Hz. The 2023 configuration resonates at 1.34 Hz with a bandwidth of 2.14 Hz. The higher-frequency resonance of the 2026 configuration is easier to suppress through conventional passive damping and is less likely to couple with road-surface harmonics at typical racing-circuit frequencies [13]. The step response of the 2026 floor shows a faster rise time (0.080 s versus 0.140 s) and higher peak overshoot (61.6% versus 45.2%), consistent with its higher natural frequency and lower damping ratio.

3.3.3 Roots Locus and Stability Margins

Root locus analysis, sweeping dF_z/dz from -200 kN/m, reveals that the critical (marginal stability) value is approximately $dF_z/dz \approx -150{,}000$ N/m (that is, $-k_z$), where the poles cross the imaginary axis. The 2026 operating point ($+15{,}134$ N/m) is positioned far from this boundary, whereas the 2023 operating point ($-86{,}373$ N/m) is significantly closer, confirming a reduced stability margin.

TABLE 7: CLASSICAL STABILITY MARGINS FROM `CONTROL_SYSTEM_ANALYSIS.PY`.

Configuration	Gain Margin (dB)	Phase Margin ($^\circ$)	k_{eff} (kN/m)
2026	137.9	99.5	165.1
2023	138.0	103.9	63.6

Both configurations exhibit near-identical, very large classical Nyquist stability margins, as expected for two stable

systems. The practically meaningful discriminators are k_{eff} and ζ, which govern oscillation amplitude and duration; the 2026 floor's higher effective stiffness requires 61% more damping to reach critical damping ($c_{\text{crit}} = 22{,}988$ versus $14{,}269\ \text{N} \cdot \text{s/m}$), further confirming that root-locus and time-domain analyses are more informative than Nyquist margins for this application.

4.0 Discussion

4.1 Physical Mechanism of Porpoising

The results of this study permit a coherent physical narrative of the porpoising instability and its mitigation. The critical stability condition is

$$k_{eff} = k_z + \left.\frac{dF_z}{dz}\right|_{z_{eq}} > 0. \tag{15}$$

Although this remains positive and therefore technically stable in the linearised sense, the modest effective stiffness produces the lower natural frequency (1.38 Hz) and generates more oscillation cycles over 3 s. In practice, transient flow effects not captured by steady-state CFD, including aerodynamic lag and hysteresis, are likely to drive the effective aerodynamic stiffness more negative than the quasi-static estimate, potentially pushing the 2023 configuration into a genuinely unstable regime.

4.2 How 2026 Regulations Mitigate Porpoising

The three geometric interventions of the 2026 regulations each address the instability mechanism through distinct physical pathways.

The flatter floor profile reduces Venturi tunnel depth and hence the sensitivity of the effective cross-sectional area to changes in ride height. When the tunnel geometry is shallower, a given vertical displacement of the car produces a smaller fractional change in flow area, resulting in a smaller velocity perturbation, a smaller pressure change, and a flatter $F_z(z)$ curve with dF_z/dz closer to zero.

The shorter Venturi tunnels reduce the spatial extent over which underbody flow is accelerated, limiting the peak velocity and hence peak suction attainable within the tunnel. This reduces both the absolute downforce magnitude at low ride heights and the steepness of stall onset. The CFD data confirm this: the 2026 floor achieves a maximum of approximately $-3{,}720$ N at 20 mm, compared to the 2023 maximum of approximately $-4{,}481$ N at 40 mm, at the cost of a substantially more favourable stability characteristic.

The lower diffuser angle (8–12° versus 15–20°) decelerates the underbody flow more gradually as it exits the rear of the floor. Classical Falkner–Skan boundary-layer theory shows that increasingly adverse pressure gradients promote separation, while weaker adverse gradients maintain attached flow [14]. Separation onset is therefore delayed to lower ride heights, and the $F_z(z)$ curve no longer exhibits the sharp inflection associated with abrupt diffuser stall.

4.3 Limitations and Sources of Error

Several limitations of the present study should be acknowledged. All simulations represent fixed-ride-height equilibria. Real porpoising involves transient flow dynamics; stall and reattachment events carry temporal lags related to the convective timescale of the diffuser flow that can amplify the effective aerodynamic force variation beyond what quasi-static analysis predicts. A dynamic mesh CFD study with time-varying ride height would capture these effects, albeit at substantially higher computational cost.

The exclusion of wheels, front wing, rear wing, and sidepods means the simulated aerodynamic forces represent only the underbody contribution. Wheel wakes in particular are known to influence diffuser inflow conditions and could modify stall characteristics. For the purposes of quantifying the relative stability improvement between the two configurations in isolation, however, this simplification is considered appropriate.

Finally, the analysis is conducted at a single freestream velocity of 65 m/s. Since the aerodynamic stiffness gradient scales with dynamic pressure, the stability margin relative to the critical threshold will change meaningfully across the speed range. A speed sweep from roughly 50 to 90 m/s would provide a more complete picture of where, if anywhere, the 2026 floor approaches its stability limit.

4.4 Recommendations

The quasi-static nature of the CFD simulations identified in Section 4.3 means that teams operating near the lower end of the 2026 ride-height envelope cannot rely on the steady-state stiffness gradient alone to confirm stability, and transient aerodynamic lag could push the effective dF_z/dz more negative than predicted. It is therefore recommended that teams conduct at minimum a ± 3 mm dynamic ride-height oscillation CFD study at their homologated equilibrium point before committing to a suspension setup, rather than relying on static tunnel data alone.

The single-speed limitation (65 m/s) means the stability margin quantified here applies only to the medium-speed regime where porpoising was historically worst. Since aerodynamic stiffness scales with dynamic pressure, the margin narrows at higher speeds. Teams should treat the 2026 floor as stable at race trim but commission a speed sweep from 50–90 m/s before approving low-speed slow-corner setups, where the dynamic pressure reduction may shift dF_z/dz unexpectedly.

The exclusion of wheel wakes from the CFD domain means the diffuser inflow conditions modelled here are cleaner than reality. Until full-car validation data are available, a conservative design margin of at least 20% above the critical dF_z/dz threshold is recommended rather than operating at the edge of the stability envelope.

From a regulatory standpoint, the FIA should encode a minimum permissible dF_z/dz at the homologated equilibrium ride height directly into the technical regulations rather than relying on geometric proxies. The geometric constraints on diffuser angle and tunnel length are necessary but not sufficient; a team could satisfy both while still producing a destabilising stiffness gradient through creative planform geometry.

5.0 Conclusions

This study applied a CFD ride-height sweep analysis and ODE-based dynamic stability modelling to quantify the aeromechanical improvements achieved by the 2026 FIA Formula 1 floor regulations relative to the 2022–2025 regulatory framework. The three primary objectives established in Section 1.3 have each been addressed.

The primary objective, quantifying the ride-height sensitivity of $F_z(z)$ and $M_y(z)$ for both configurations, was achieved through a six-point CFD sweep at $z = 15$–40 mm using a steady-state RANS solver with k–ω SST turbulence closure. The 2026 floor yields a nearly flat $F_z(z)$ relationship with a total downforce variation of only ≈ 260 N across the full sweep, compared to $\approx 2{,}600$ N for the 2023 floor. The aerodynamic stiffness gradient at the equilibrium ride height of 25 mm shifts from $-86{,}373$ N/m (2023) to $+15{,}134$ N/m (2026), a reduction of 82.5%. This converts the aerodynamic contribution to the effective system stiffness from a strongly destabilizing negative spring to a mildly stabilizing one, directly eliminating the primary aeromechanical driver of porpoising.

The secondary objective, coupling the CFD-extracted stiffness data to a 2-DOF ODE vehicle dynamics model, was fulfilled through the linearized heave-pitch formulation. Effective system stiffness increases from 63.6 kN/m to 165.1 kN/m, raising the primary natural frequency from 1.38 Hz to 2.26 Hz. The damping ratio decreases from $\zeta = 0.245$ to $\zeta = 0.152$ as a consequence of the higher stiffness; however, the increased oscillation frequency produces only 6 perturbation cycles over 3 s compared to 8 for the 2023 floor, indicating faster real-time energy dissipation. Both configurations settle to within 2% of equilibrium from a -5 mm heave disturbance within approximately 1.76 s, confirming that both are dynamically stable under the quasi-static assumptions of this analysis.

The tertiary objective, assessing frequency-domain behaviour and stability margins through control system methods, was addressed via transfer function derivation, Routh-Hurwitz analysis, root locus, Bode, and Nyquist analyses. Root locus analysis confirms that the 2026 operating point ($dF_z/dz = +15{,}134$ N/m) is positioned far from the marginal stability boundary at $dF_z/dz \approx -150{,}000$ N/m, whereas the 2023 operating point ($-86{,}373$ N/m) lies significantly closer.

These findings validate the FIA's design intent for the 2026 regulations. The three geometric interventions: a flatter floor profile, shorter Venturi tunnels, and a reduced diffuser expansion angle of 8–12° work through distinct but complementary physical pathways to eliminate the negative spring mechanism responsible for ground-effect porpoising. The practical consequences for teams are significant: the 2026 floor permits higher and more flexible ride height selection, softer suspension tuning on purely mechanical grounds, and a direct reduction in driver exposure to injurious whole-body vibration.

Future work should prioritize transient CFD with dynamic mesh motion to capture stall lag and aerodynamic hysteresis effects that steady-state analysis cannot resolve. Full-car aerodynamic modelling incorporating wheel wakes, front wing interactions, and cooling duct flows would provide absolute force magnitudes representative of a real vehicle. A speed sweep from 50–90 m/s is recommended to show how the stability margin evolves across the operating envelope, given that the aerodynamic stiffness gradient scales with dynamic pressure. Finally, a multi-DOF vehicle dynamics model incorporating roll, lateral load transfer, and four-corner heave coupling, validated against driver vibration dose metrics, would give the most complete characterization of driver health outcomes under both regulatory frameworks.

References

[1] FIA, "2022 FORMULA 1 TECHNICAL REGULATIONS," Feb. 2021. [Online]. Available: https://www.fia.com/sites/default/files/2022_formula_1_technical_regulations_-_iss_3_-_2021-02-19.pdf

[2] "WATCH: The origins of ground effect | Formula 1®." [Online]. Available: https://www.formula1.com/en/latest/article/watch-the-origins-of-ground-effect.6i1MZBs5xe6cjO2deb57hV

[3] "Formula 1 aerodynamics: What are Venturi tunnels and how does porpoising affect F1 cars? | Raceteq." [Online]. Available: https://www.raceteq.com/articles/2024/08/venturi-tunnels-explainer

[4] "Why F1 cars have wings – with a McLaren Aero Specialist." [Online]. Available: https://www.formula1.com/en/latest/article/f1-explains-downforce-and-why-f1-cars-have-wings-with-mclaren-aero.10xj8CJPz8s7CmwfoyWmCU

[5] "What is 'porpoising' – and why is it causing the F1 teams a headache at 2022 pre-season running? | Formula 1®." [Online]. Available: https://www.formula1.com/en/latest/article/analysis-what-is-porpoising-and-why-is-it-causing-the-f1-teams-a-headache-at.6O93LlvFMyY81L9LfSutcS

[6] J. Liang, "The Influence of F1 "porpoising" Phenomenon and Venturi Floor Design on the Performance of Racing Cars," vol. 1, no. 6, Dec. 2025.

[7] "Porpoising cars could fracture drivers' spines, warns F1 physio - Motor Sport Magazine." [Online]. Available: https://www.motorsportmagazine.com/articles/single-seaters/f1/porpoising-cars-could-fracture-drivers-spines-warns-f1-physio/

[8] "Azerbaijan GP: Lewis Hamilton reveals back pain caused by bouncing Mercedes left him 'praying for race to end'." [Online]. Available: https://www.skysports.com/f1/news/12433/12632603/azerbaijan-gp-lewis-hamilton-reveals-back-pain-caused-by-bouncing-mercedes-left-him-praying-for-race-to-end

[9] G. Richards, "FIA sticks to guns on F1 'bouncing' rule change despite top teams' objections," *The Guardian*, Aug. 2022. [Online]. Available: https://www.theguardian.com/sport/2022/aug/26/red-bull-ferrari-object-but-fia-stick-to-guns-on-new-long-term-head-trauma-rule

[10] FIA, "SECTION C : TECHNICAL REGULATIONS," Oct. 2024. [Online]. Available: https://www.fia.com/sites/default/files/fia_2026_f1_regulations_-_section_c_technical_-_iss09_-_2024-10-17.pdf

[11] "SimScale Documentation | Online Simulation Software | SimScale." [Online]. Available: https://www.simscale.com/docs/

[12] R. Rizqyandra, "CHOOSING BETWEEN K-EPSILON AND K-OMEGA FOR YOUR CFD SIMULATION: A PRACTICAL GUIDE," Feb. 2025. [Online]. Available: https://medium.com/@riqy.sci/choosing-between-k-epsilon-and-k-omega-for-your-cfd-simulation-a-practical-guide-1546d95596ff

[13] G. B. Khandavalli, "Roads — How are they described and modeled in suspension design?" Apr. 2026. [Online]. Available: https://gkhandavalli.medium.com/roads-how-are-they-described-and-modeled-in-suspension-design-39bf729f3ec8

[14] J. Dey, "A simple technique for approximate solutions of the Falkner-Skan equation," *Acta Mechanica*, vol. 77, no. 3, pp. 299–305, May 1989. [Online]. Available: https://doi.org/10.1007/BF01178328

A Raw CFD Data

The complete raw CFD output for all 12 simulations is presented in Table 8 in Section 3.1.1. This appendix provides additional tabulated data including the drag force F_x, lateral force F_y, roll moment M_x, and yaw moment M_z, which were not the primary focus of the stability analysis but are included for completeness. The near-zero lateral forces confirm the symmetry of the simulation domain and the validity of the computational setup.

TABLE 8: COMPLETE RAW CFD AERODYNAMIC DATA: ALL FORCE AND MOMENT COMPONENTS

Config	z (mm)	F_z (N)	F_x (N)	F_y (N)	M_x (N·m)	M_y (N·m)	M_z (N·m)
2026	15	−3635.5	743.0	0.43	−21.93	−8.34	−2175.6
2026	20	−3720.1	762.3	2.33	0.26	−10.45	−2168.5
2026	25	−3644.5	741.7	0.80	0.13	−10.36	−2107.8
2026	30	−3586.6	747.0	4.11	3.75	−2.48	−2095.3
2026	35	−3482.6	733.5	2.42	−3.06	−5.37	−2008.6
2026	40	−3458.2	733.4	0.46	−4.27	−10.10	−1940.1
2023	15	−1855.5	1313.3	−0.94	23.24	−3.06	−3921.1
2023	20	−2531.4	1318.5	−23.00	22.87	−2.25	−3753.2
2023	25	−2902.8	1341.0	14.03	−44.61	−80.89	−3554.5
2023	30	−3358.8	1373.4	17.30	−32.66	−64.72	−3331.1
2023	35	−3896.8	1373.5	0.30	−5.52	−3.89	−3165.4
2023	40	−4481.4	1383.9	−7.74	−11.81	−61.46	−2967.8

B Python Source Code

The complete Python source code used for polynomial fitting, dynamic simulation, and control-system analysis is provided below. The scripts were executed in Python 3.11 using NumPy, SciPy, and Matplotlib. Two source files are included: one for the coupled aerodynamic-dynamic analysis workflow and one for the dedicated control-systems study.

Listing 1: Source code for the coupled aerodynamic-dynamic analysis workflow

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

print("="*80)
print("COMPREHENSIVE F1 AERODYNAMIC-DYNAMIC COUPLING ANALYSIS")
print("Coupled Heave-Pitch ODE System with Full Stability & Modal Analysis")
print("="*80)

# SECTION 1: CFD DATA & POLYNOMIAL FITTING

print("\n[1] Loading CFD Data and Fitting Polynomial Functions...")

# Ride heights (m) - CFD data points
z_data_26 = np.array([15, 20, 25, 30, 35, 40]) / 1000
z_data_23 = np.array([15, 20, 25, 30, 35, 40]) / 1000
```

```
17
18  # Vertical force (downforce, negative = downward) - CFD data
19  Fz_data_26 = np.array([-3635.48, -3720.09, -3644.50, -3586.64, -3482.62, -3458.16])
20  Fz_data_23 = np.array([-1855.47, -2531.38, -2902.84, -3358.83, -3896.79, -4481.41])
21
22  # Pitch moment (about lateral axis, y-axis) - CFD data
23  My_data_26 = np.array([-8.344, -10.451, -10.362, -2.475, -5.370, -10.096])
24  My_data_23 = np.array([-3.057, -2.251, -80.892, -64.724, -3.891, -61.460])
25
26  # Fit cubic polynomials (higher order captures nonlinearity)
27  poly_Fz_26 = np.polyfit(z_data_26, Fz_data_26, 3)
28  poly_Fz_23 = np.polyfit(z_data_23, Fz_data_23, 3)
29  poly_My_26 = np.polyfit(z_data_26, My_data_26, 3)
30  poly_My_23 = np.polyfit(z_data_23, My_data_23, 3)
31
32  # Create polynomial functions
33  Fz26_func = np.poly1d(poly_Fz_26)
34  Fz23_func = np.poly1d(poly_Fz_23)
35  My26_func = np.poly1d(poly_My_26)
36  My23_func = np.poly1d(poly_My_23)
37
38  # Derivatives (aerodynamic stiffness)
39  dFz26_dz = np.polyder(Fz26_func)
40  dFz23_dz = np.polyder(Fz23_func)
41  dMy26_dz = np.polyder(My26_func)
42  dMy23_dz = np.polyder(My23_func)
43
44  # Second derivatives (aerodynamic nonlinearity)
45  d2Fz26_dz2 = np.polyder(dFz26_dz)
46  d2Fz23_dz2 = np.polyder(dFz23_dz)
47
48  print("[OK] Cubic polynomials fitted to CFD data")
49  print("[OK] First and second derivatives computed")
50
51
52  # SECTION 2: VEHICLE PARAMETERS (Realistic F1 values)
53
54  print("\n[2] Defining Vehicle Physical Parameters...")
55
56  # Mass properties
57  m = 800.0
58  I_pitch = 1200.0
59  L = 3.6
60  a = 1.6
61  b = 2.0
62
63  # Suspension properties (combined front + rear)
64  kz = 150000.0
65  cz = 3500.0
66  k_pitch = 80000.0
67  c_pitch = 5000.0
68
69  # Operating point
70  z_eq = 0.025
71  v = 65.0
72
73  print(f"  Mass: {m} kg")
74  print(f"  Pitch inertia: {I_pitch} kg*m^2")
75  print(f"  Heave stiffness: {kz/1000:.0f} kN/m")
76  print(f"  Pitch stiffness: {k_pitch/1000:.0f} kNm/rad")
77
78  # SECTION 3: EQUILIBRIUM ANALYSIS
79
80  print("\n[3] Computing Equilibrium State...")
81
```

```
Fz26_eq = Fz26_func(z_eq)
Fz23_eq = Fz23_func(z_eq)
My26_eq = My26_func(z_eq)
My23_eq = My23_func(z_eq)

print(f"  At z = {z_eq*1000} mm:")
print(f"    2026: Fz = {Fz26_eq:.1f} N, My = {My26_eq:.1f} Nm")
print(f"    2023: Fz = {Fz23_eq:.1f} N, My = {My23_eq:.1f} Nm")

# SECTION 4: LINEARIZED STABILITY ANALYSIS

print("\n[4] Linearized Stability Analysis at Equilibrium...")

# Aerodynamic derivatives at equilibrium
dFz26_eq = dFz26_dz(z_eq)
dFz23_eq = dFz23_dz(z_eq)
dMy26_eq = dMy26_dz(z_eq)
dMy23_eq = dMy23_dz(z_eq)

print(f"  Aerodynamic stiffness contributions:")
print(f"    2026: dFz/dz = {dFz26_eq:.0f} N/m, dMy/dz = {dMy26_eq:.1f} Nm/m")
print(f"    2023: dFz/dz = {dFz23_eq:.0f} N/m, dMy/dz = {dMy23_eq:.1f} Nm/m")

def linearized_system_matrices(dFz_dz, dMy_dz):
    """
    Construct linearized state-space matrices for coupled heave-pitch system.

    State vector: X = [z, z_dot, theta, theta_dot]^T

    Equations of motion (linearized):
      m*z_ddot + cz*z_dot + (kz + dFz/dz)*z = 0
      I*theta_ddot + c_pitch*theta_dot + (k_pitch + dMy/dz)*theta = 0

    State-space form: X_dot = A*X
    """
    # Effective stiffness (mechanical + aerodynamic)
    k_eff_z = kz + dFz_dz  # Can be negative if aero destabilizes
    k_eff_pitch = k_pitch + dMy_dz

    # State-space matrix A (4x4)
    A = np.array([
        [0,           1,              0,              0],
        [-k_eff_z/m,  -cz/m,          0,              0],
        [0,           0,              0,              1],
        [0,           0,              -k_eff_pitch/I_pitch, -c_pitch/I_pitch]
    ])

    return A, k_eff_z, k_eff_pitch

A_26, k_eff_z_26, k_eff_pitch_26 = linearized_system_matrices(dFz26_eq, dMy26_eq)
A_23, k_eff_z_23, k_eff_pitch_23 = linearized_system_matrices(dFz23_eq, dMy23_eq)

print(f"\n  Effective stiffness (mechanical + aero):")
print(f"    2026 heave:  k_eff = {k_eff_z_26/1000:.1f} kN/m")
print(f"    2023 heave:  k_eff = {k_eff_z_23/1000:.1f} kN/m")
print(f"    2026 pitch:  k_eff = {k_eff_pitch_26/1000:.1f} kNm/rad")
print(f"    2023 pitch:  k_eff = {k_eff_pitch_23/1000:.1f} kNm/rad")

# Eigenvalue analysis
eigenvals_26, eigenvecs_26 = np.linalg.eig(A_26)
eigenvals_23, eigenvecs_23 = np.linalg.eig(A_23)

print(f"\n  Eigenvalue Analysis (? = ? ? jomega):")
print(f"    2026 eigenvalues:")
for i, lam in enumerate(eigenvals_26):
```

```
    if np.isreal(lam):
        print(f"      ?{i+1} = {lam.real:.3f} (real)")
    else:
        print(f"      ?{i+1} = {lam.real:.3f} ? j{abs(lam.imag):.3f}")

print(f"    2023 eigenvalues:")
for i, lam in enumerate(eigenvals_23):
    if np.isreal(lam):
        print(f"      ?{i+1} = {lam.real:.3f} (real)")
    else:
        print(f"      ?{i+1} = {lam.real:.3f} ? j{abs(lam.imag):.3f}")

# Stability check
stable_26 = all(np.real(eigenvals_26) < 0)
stable_23 = all(np.real(eigenvals_23) < 0)

print(f"\n  Stability verdict:")
print(f"    2026: {'STABLE' if stable_26 else 'UNSTABLE'} (all Re(?) < 0: {stable_26})")
print(f"    2023: {'STABLE' if stable_23 else 'UNSTABLE'} (all Re(?) < 0: {stable_23})")

# Natural frequencies and damping ratios
def extract_modal_params(eigenvals):
    """Extract natural frequency and damping ratio from complex conjugate pairs"""
    modes = []
    processed = set()

    for i, lam in enumerate(eigenvals):
        if i in processed:
            continue

        if np.iscomplex(lam):
            sigma = np.real(lam)
            omega_d = abs(np.imag(lam))  # Damped natural frequency
            omega_n = np.sqrt(sigma**2 + omega_d**2)  # Undamped natural frequency
            zeta = -sigma / omega_n if omega_n > 0 else 0  # Damping ratio

            # Find conjugate pair
            for j, lam2 in enumerate(eigenvals):
                if j > i and np.isclose(lam, np.conj(lam2)):
                    processed.add(j)
                    break

            modes.append({
                'omega_n': omega_n,
                'omega_d': omega_d,
                'zeta': zeta,
                'freq_hz': omega_d / (2*np.pi),
                'period': 2*np.pi/omega_d if omega_d > 0 else np.inf
            })

    return modes

modes_26 = extract_modal_params(eigenvals_26)
modes_23 = extract_modal_params(eigenvals_23)

print(f"\n  Modal Parameters:")
print(f"    2026:")
for i, mode in enumerate(modes_26):
    print(f"      Mode {i+1}: omega_n = {mode['omega_n']:.2f} rad/s, f = {mode['freq_hz']:.2f} Hz, zeta = {
     mode['zeta']:.3f}")
print(f"    2023:")
for i, mode in enumerate(modes_23):
    print(f"      Mode {i+1}: omega_n = {mode['omega_n']:.2f} rad/s, f = {mode['freq_hz']:.2f} Hz, zeta = {
     mode['zeta']:.3f}")

```

```python
# SECTION 5: NONLINEAR ODE SIMULATION

print("\n[5] Nonlinear Time-Domain Simulation...")

def coupled_heave_pitch_ode(t, X, Fz_func, My_func, Fz_eq, My_eq):
    """
    Full nonlinear coupled ODE system.

    State: X = [z, z_dot, theta, theta_dot]
    where z is heave displacement from equilibrium, theta is pitch angle

    Equations:
      m*z_ddot = Fz(z_eq + z) - Fz_eq - cz*z_dot - kz*z
      I*theta_ddot = My(z_eq + z) - My_eq - c_pitch*theta_dot - k_pitch*theta
    """
    z, z_dot, theta, theta_dot = X

    # Current ride height
    z_current = z_eq + z

    # Aerodynamic forces (perturbation from equilibrium)
    Fz_aero = Fz_func(z_current) - Fz_eq
    My_aero = My_func(z_current) - My_eq

    # Accelerations
    z_ddot = (Fz_aero - cz*z_dot - kz*z) / m
    theta_ddot = (My_aero - c_pitch*theta_dot - k_pitch*theta) / I_pitch

    return [z_dot, z_ddot, theta_dot, theta_ddot]

# Initial conditions: 5mm heave perturbation + 0.5 deg pitch
z0 = -0.005  # -5 mm (downward)
theta0 = 0.01  # ~0.57 deg
X0 = [z0, 0.0, theta0, 0.0]

# Time span
t_span = (0, 3.0)
t_eval = np.linspace(0, 3.0, 3000)

print(f"  Initial conditions: z0 = {z0*1000:.1f} mm, theta0 = {np.degrees(theta0):.2f} deg")
print(f"  Simulating 3 seconds...")

# Solve for both configurations
sol_26 = solve_ivp(
    coupled_heave_pitch_ode, t_span, X0, t_eval=t_eval,
    args=(Fz26_func, My26_func, Fz26_eq, My26_eq),
    method='RK45', rtol=1e-6
)

sol_23 = solve_ivp(
    coupled_heave_pitch_ode, t_span, X0, t_eval=t_eval,
    args=(Fz23_func, My23_func, Fz23_eq, My23_eq),
    method='RK45', rtol=1e-6
)

print(f"  [OK] Integration complete")

# Extract solutions
z_26 = sol_26.y[0] * 1000  # Convert to mm
z_23 = sol_23.y[0] * 1000
theta_26 = np.degrees(sol_26.y[2])  # Convert to degrees
theta_23 = np.degrees(sol_23.y[2])


# SECTION 6: ENERGY ANALYSIS
```

```python

print("\n[6] Energy Analysis...")

def compute_energy(sol, Fz_func, My_func, Fz_eq, My_eq):
    """Compute kinetic, potential, and total energy"""
    z = sol.y[0]
    z_dot = sol.y[1]
    theta = sol.y[2]
    theta_dot = sol.y[3]

    # Kinetic energy
    KE_heave = 0.5 * m * z_dot**2
    KE_pitch = 0.5 * I_pitch * theta_dot**2
    KE_total = KE_heave + KE_pitch

    # Potential energy (mechanical + aerodynamic)
    PE_mech = 0.5 * kz * z**2 + 0.5 * k_pitch * theta**2

    # Aerodynamic potential (integrate force from equilibrium)
    PE_aero = np.zeros_like(z)
    for i in range(len(z)):
        z_current = z_eq + z[i]
        # Simplified: use average force over displacement
        PE_aero[i] = (Fz_func(z_current) - Fz_eq) * z[i]

    PE_total = PE_mech + PE_aero
    E_total = KE_total + PE_total

    return KE_total, PE_total, E_total

KE_26, PE_26, E_26 = compute_energy(sol_26, Fz26_func, My26_func, Fz26_eq, My26_eq)
KE_23, PE_23, E_23 = compute_energy(sol_23, Fz23_func, My23_func, Fz23_eq, My23_eq)

# Energy dissipation rate
energy_loss_26 = (E_26[0] - E_26[-1]) / E_26[0] * 100
energy_loss_23 = (E_23[0] - E_23[-1]) / E_23[0] * 100

print(f"  Initial total energy:")
print(f"    2026: {E_26[0]:.1f} J")
print(f"    2023: {E_23[0]:.1f} J")
print(f"  Energy dissipated (3 sec):")
print(f"    2026: {energy_loss_26:.1f}% (well-damped)")
print(f"    2023: {energy_loss_23:.1f}% (poorly-damped)")


# SECTION 7: PERFORMANCE METRICS

print("\n[7] Performance Metrics...")

# Settling time (time to reach 2% of initial value)
def settling_time(t, response, threshold=0.02):
    """Find time when |response| < threshold * |initial|"""
    initial = abs(response[0])
    for i in range(len(response)-1, 0, -1):
        if abs(response[i]) > threshold * initial:
            return t[i] if i < len(t)-1 else t[-1]
    return 0

ts_heave_26 = settling_time(sol_26.t, z_26)
ts_heave_23 = settling_time(sol_23.t, z_23)
ts_pitch_26 = settling_time(sol_26.t, theta_26)
ts_pitch_23 = settling_time(sol_23.t, theta_23)

print(f"  Settling time (2% threshold):")
print(f"    2026 heave: {ts_heave_26:.3f} s, pitch: {ts_pitch_26:.3f} s")
```

```
print(f"    2023 heave: {ts_heave_23:.3f} s, pitch: {ts_pitch_23:.3f} s")

# Peak overshoot
overshoot_26 = (max(abs(z_26)) - abs(z_26[0])) / abs(z_26[0]) * 100
overshoot_23 = (max(abs(z_23)) - abs(z_23[0])) / abs(z_23[0]) * 100

print(f"  Peak overshoot:")
print(f"    2026: {overshoot_26:.1f}%")
print(f"    2023: {overshoot_23:.1f}%")

# Number of oscillations
def count_zero_crossings(signal):
    """Count zero crossings (oscillations)"""
    return np.sum(np.diff(np.sign(signal)) != 0) // 2

osc_26 = count_zero_crossings(z_26)
osc_23 = count_zero_crossings(z_23)

print(f"  Oscillation cycles (3 sec):")
print(f"    2026: {osc_26} cycles")
print(f"    2023: {osc_23} cycles")

# SECTION 8: SYMBOLIC DERIVATION (For report documentation)

print("\n[8] Symbolic Mathematical Derivation...")

print("  Equations of motion (symbolic):")
print("\n  Heave equation:")
print("    m * d2z/dt2 = F_z - c_z * dz/dt - k_z * z")

print("\n  Pitch equation:")
print("    I * d2theta/dt2 = M_y - c_theta * dtheta/dt - k_theta * theta")

# Linearization
print("\n  Linearized aerodynamic forces:")
print(f"    F_z(z) ˜= F_z(z_eq) + (dF_z/dz)|_eq * ?z")
print(f"    M_y(z) ˜= M_y(z_eq) + (dM_y/dz)|_eq * ?z")

print("\n  State-space representation:")
print("    X = [z, ?, theta, theta?]?")
print("    ? = A*X where:")
print("    A = | 0             1             0                 0           |")
print("        | -k_eff/m      -c_z/m        0                 0           |")
print("        | 0             0             0                 1           |")
print("        | 0             0             -k_eff_theta/I        -c_theta/I      |")
print(f"\n    k_eff = k_z + dF_z/dz  (aerodynamic stiffness adds to mechanical)")

# SECTION 9: COMPREHENSIVE VISUALIZATION

print("\n[9] Generating Comprehensive Plots...")

# Create figure with multiple subplots
fig = plt.figure(figsize=(16, 12))

# Plot 1: Downforce vs Ride Height
ax1 = plt.subplot(3, 3, 1)
z_plot = np.linspace(0.015, 0.04, 200)
ax1.plot(z_plot*1000, Fz26_func(z_plot), 'b-', linewidth=2, label='2026 fit')
ax1.plot(z_plot*1000, Fz23_func(z_plot), 'r-', linewidth=2, label='2023 fit')
ax1.scatter(z_data_26*1000, Fz_data_26, color='blue', marker='o', s=80, edgecolors='black', linewidth=1.5,
    zorder=5)
ax1.scatter(z_data_23*1000, Fz_data_23, color='red', marker='s', s=80, edgecolors='black', linewidth=1.5,
    zorder=5)
ax1.axvline(25, color='gray', linestyle='--', alpha=0.7, label='Equilibrium')
ax1.set_xlabel('Ride Height (mm)', fontsize=11, fontweight='bold')
```

```
ax1.set_ylabel('Vertical Force $F_z$ (N)', fontsize=11, fontweight='bold')
ax1.set_title('Downforce vs Ride Height', fontsize=12, fontweight='bold')
ax1.legend(frameon=True, shadow=True)
ax1.grid(True, alpha=0.3)

# Plot 2: Aerodynamic Stiffness Gradient
ax2 = plt.subplot(3, 3, 2)
ax2.plot(z_plot*1000, dFz26_dz(z_plot), 'b-', linewidth=2, label='2026 dF_z/dz')
ax2.plot(z_plot*1000, dFz23_dz(z_plot), 'r-', linewidth=2, label='2023 dF_z/dz')
ax2.axhline(0, color='black', linestyle='-', linewidth=0.8)
ax2.axvline(25, color='gray', linestyle='--', alpha=0.7)
ax2.fill_between(z_plot*1000, dFz23_dz(z_plot), alpha=0.2, color='red', label='Destabilizing region')
ax2.set_xlabel('Ride Height (mm)', fontsize=11, fontweight='bold')
ax2.set_ylabel('dF_z/dz (N/m)', fontsize=11, fontweight='bold')
ax2.set_title('Aerodynamic Stiffness Gradient', fontsize=12, fontweight='bold')
ax2.legend(frameon=True, shadow=True)
ax2.grid(True, alpha=0.3)

# Plot 3: Pitch Moment
ax3 = plt.subplot(3, 3, 3)
ax3.plot(z_plot*1000, My26_func(z_plot), 'b-', linewidth=2, label='2026 fit')
ax3.plot(z_plot*1000, My23_func(z_plot), 'r-', linewidth=2, label='2023 fit')
ax3.scatter(z_data_26*1000, My_data_26, color='blue', marker='o', s=80, edgecolors='black', linewidth=1.5,
    zorder=5)
ax3.scatter(z_data_23*1000, My_data_23, color='red', marker='s', s=80, edgecolors='black', linewidth=1.5,
    zorder=5)
ax3.axvline(25, color='gray', linestyle='--', alpha=0.7)
ax3.set_xlabel('Ride Height (mm)', fontsize=11, fontweight='bold')
ax3.set_ylabel('Pitch Moment $M_y$ (N*m)', fontsize=11, fontweight='bold')
ax3.set_title('Pitch Moment vs Ride Height', fontsize=12, fontweight='bold')
ax3.legend(frameon=True, shadow=True)
ax3.grid(True, alpha=0.3)

# Plot 4: Heave Response
ax4 = plt.subplot(3, 3, 4)
ax4.plot(sol_26.t, z_26, 'b-', linewidth=2, label='2026', alpha=0.8)
ax4.plot(sol_23.t, z_23, 'r-', linewidth=2, label='2023', alpha=0.8)
ax4.axhline(0, color='black', linestyle='-', linewidth=0.8)
ax4.set_xlabel('Time (s)', fontsize=11, fontweight='bold')
ax4.set_ylabel('Heave Displacement (mm)', fontsize=11, fontweight='bold')
ax4.set_title('Heave Response to Perturbation', fontsize=12, fontweight='bold')
ax4.legend(frameon=True, shadow=True)
ax4.grid(True, alpha=0.3)

# Plot 5: Pitch Response
ax5 = plt.subplot(3, 3, 5)
ax5.plot(sol_26.t, theta_26, 'b-', linewidth=2, label='2026', alpha=0.8)
ax5.plot(sol_23.t, theta_23, 'r-', linewidth=2, label='2023', alpha=0.8)
ax5.axhline(0, color='black', linestyle='-', linewidth=0.8)
ax5.set_xlabel('Time (s)', fontsize=11, fontweight='bold')
ax5.set_ylabel('Pitch Angle (degrees)', fontsize=11, fontweight='bold')
ax5.set_title('Pitch Response to Perturbation', fontsize=12, fontweight='bold')
ax5.legend(frameon=True, shadow=True)
ax5.grid(True, alpha=0.3)

# Plot 6: Phase Portrait (Heave)
ax6 = plt.subplot(3, 3, 6)
ax6.plot(z_26, sol_26.y[1]*1000, 'b-', linewidth=2, label='2026', alpha=0.7)
ax6.plot(z_23, sol_23.y[1]*1000, 'r-', linewidth=2, label='2023', alpha=0.7)
ax6.scatter([z_26[0]], [sol_26.y[1][0]*1000], color='green', marker='o', s=100, zorder=5, label='Start')
ax6.scatter([0], [0], color='black', marker='x', s=100, zorder=5, label='Equilibrium')
ax6.set_xlabel('Heave Position (mm)', fontsize=11, fontweight='bold')
ax6.set_ylabel('Heave Velocity (mm/s)', fontsize=11, fontweight='bold')
ax6.set_title('Heave Phase Portrait', fontsize=12, fontweight='bold')
ax6.legend(frameon=True, shadow=True)
```

```
ax6.grid(True, alpha=0.3)

# Plot 7: Energy Evolution
ax7 = plt.subplot(3, 3, 7)
ax7.plot(sol_26.t, E_26, 'b-', linewidth=2, label='2026 Total', alpha=0.8)
ax7.plot(sol_23.t, E_23, 'r-', linewidth=2, label='2023 Total', alpha=0.8)
ax7.plot(sol_26.t, KE_26, 'b--', linewidth=1.5, label='2026 Kinetic', alpha=0.5)
ax7.plot(sol_23.t, KE_23, 'r--', linewidth=1.5, label='2023 Kinetic', alpha=0.5)
ax7.set_xlabel('Time (s)', fontsize=11, fontweight='bold')
ax7.set_ylabel('Energy (J)', fontsize=11, fontweight='bold')
ax7.set_title('Energy Evolution', fontsize=12, fontweight='bold')
ax7.legend(frameon=True, shadow=True, fontsize=9)
ax7.grid(True, alpha=0.3)

# Plot 8: Eigenvalue Plot (Complex Plane)
ax8 = plt.subplot(3, 3, 8)
for i, lam in enumerate(eigenvals_26):
    ax8.scatter(np.real(lam), np.imag(lam), color='blue', marker='o', s=150,
                edgecolors='black', linewidth=2, zorder=5, label='2026' if i==0 else '')
for i, lam in enumerate(eigenvals_23):
    ax8.scatter(np.real(lam), np.imag(lam), color='red', marker='s', s=150,
                edgecolors='black', linewidth=2, zorder=5, label='2023' if i==0 else '')
ax8.axvline(0, color='black', linestyle='-', linewidth=1.5, label='Stability boundary')
ax8.axhline(0, color='gray', linestyle='--', linewidth=0.8)
ax8.set_xlabel('Real Part ? (1/s)', fontsize=11, fontweight='bold')
ax8.set_ylabel('Imaginary Part omega (rad/s)', fontsize=11, fontweight='bold')
ax8.set_title('Eigenvalue Locations (Stability Map)', fontsize=12, fontweight='bold')
ax8.legend(frameon=True, shadow=True)
ax8.grid(True, alpha=0.3)
ax8.text(0.02, 0.98, 'Unstable ?', transform=ax8.transAxes, ha='left', va='top',
         fontsize=10, bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.3))

# Plot 9: Second Derivative (Nonlinearity)
ax9 = plt.subplot(3, 3, 9)
ax9.plot(z_plot*1000, d2Fz26_dz2(z_plot), 'b-', linewidth=2, label='2026 d^2F_z/dz^2')
ax9.plot(z_plot*1000, d2Fz23_dz2(z_plot), 'r-', linewidth=2, label='2023 d^2F_z/dz^2')
ax9.axhline(0, color='black', linestyle='-', linewidth=0.8)
ax9.axvline(25, color='gray', linestyle='--', alpha=0.7)
ax9.set_xlabel('Ride Height (mm)', fontsize=11, fontweight='bold')
ax9.set_ylabel('d^2F_z/dz^2 (N/m^2)', fontsize=11, fontweight='bold')
ax9.set_title('Aerodynamic Nonlinearity', fontsize=12, fontweight='bold')
ax9.legend(frameon=True, shadow=True)
ax9.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('comprehensive_analysis.png', dpi=300, bbox_inches='tight')
print("  [OK] Saved: comprehensive_analysis.png")


# SECTION 10: SUMMARY REPORT

print("\n" + "="*80)
print("SUMMARY REPORT")
print("="*80)

print("\n1. AERODYNAMIC SENSITIVITY REDUCTION:")
print(f"   ? Gradient at equilibrium reduced by {abs((dFz26_eq - dFz23_eq)/dFz23_eq)*100:.1f}%")
print(f"     (2023: {dFz23_eq:.0f} N/m ? 2026: {dFz26_eq:.0f} N/m)")

print("\n2. STABILITY IMPROVEMENT:")
print(f"   ? 2026: {('STABLE' if stable_26 else 'UNSTABLE')} with zeta = {modes_26[0]['zeta']:.3f}")
print(f"   ? 2023: {('STABLE' if stable_23 else 'UNSTABLE')} with zeta = {modes_23[0]['zeta']:.3f}")
print(f"   ? Damping ratio increased by {(modes_26[0]['zeta'] - modes_23[0]['zeta'])/modes_23[0]['zeta
    ']*100:.1f}%")

```

```
print("\n3. DYNAMIC PERFORMANCE:")
print(f"   ? Settling time: 2026 = {ts_heave_26:.2f}s vs 2023 = {ts_heave_23:.2f}s")
print(f"   ? Peak overshoot: 2026 = {overshoot_26:.1f}% vs 2023 = {overshoot_23:.1f}%")
print(f"   ? Energy dissipation: 2026 = {energy_loss_26:.1f}% vs 2023 = {energy_loss_23:.1f}%")

print("\n4. PORPOISING ASSESSMENT:")
print(f"   ? 2026 natural frequency: {modes_26[0]['freq_hz']:.2f} Hz (period: {modes_26[0]['period']:.3f}s)")
print(f"   ? 2023 natural frequency: {modes_23[0]['freq_hz']:.2f} Hz (period: {modes_23[0]['period']:.3f}s)")
print(f"   ? Oscillation cycles in 3s: 2026 = {osc_26} vs 2023 = {osc_23}")


print("\n" + "="*80)
print("Analysis complete. All plots saved")
print("="*80)

# Save numerical results to file
results_summary = f"""
COMPREHENSIVE F1 AERODYNAMIC-DYNAMIC COUPLING ANALYSIS

CONFIGURATION COMPARISON: 2023 vs 2026 Regulations

1. AERODYNAMIC LOADS AT EQUILIBRIUM (z = 25 mm)
   2026: Fz = {Fz26_eq:.1f} N, My = {My26_eq:.1f} Nm
   2023: Fz = {Fz23_eq:.1f} N, My = {My23_eq:.1f} Nm

2. AERODYNAMIC STIFFNESS (dFz/dz at equilibrium)
   2026: {dFz26_eq:.0f} N/m
   2023: {dFz23_eq:.0f} N/m
   Reduction: {abs((dFz26_eq - dFz23_eq)/dFz23_eq)*100:.1f}%

3. EFFECTIVE SYSTEM STIFFNESS
   2026: k_eff = {k_eff_z_26/1000:.1f} kN/m
   2023: k_eff = {k_eff_z_23/1000:.1f} kN/m

4. MODAL PARAMETERS
   2026 Heave Mode:
     - Natural frequency: {modes_26[0]['omega_n']:.2f} rad/s ({modes_26[0]['freq_hz']:.2f} Hz)
     - Damping ratio: {modes_26[0]['zeta']:.3f}
     - Period: {modes_26[0]['period']:.3f} s

   2023 Heave Mode:
     - Natural frequency: {modes_23[0]['omega_n']:.2f} rad/s ({modes_23[0]['freq_hz']:.2f} Hz)
     - Damping ratio: {modes_23[0]['zeta']:.3f}
     - Period: {modes_23[0]['period']:.3f} s

5. EIGENVALUES
   2026: {[f'{np.real(l):.3f}?j{np.imag(l):.3f}' for l in eigenvals_26]}
   2023: {[f'{np.real(l):.3f}?j{np.imag(l):.3f}' for l in eigenvals_23]}

6. STABILITY VERDICT
   2026: {'STABLE' if stable_26 else 'UNSTABLE'}
   2023: {'STABLE' if stable_23 else 'UNSTABLE'}

7. TIME-DOMAIN PERFORMANCE (Initial: z0=-5mm, theta0=0.57 deg)
   Settling time (2%):
     2026: {ts_heave_26:.3f} s (heave), {ts_pitch_26:.3f} s (pitch)
     2023: {ts_heave_23:.3f} s (heave), {ts_pitch_23:.3f} s (pitch)

   Peak overshoot:
     2026: {overshoot_26:.1f}%
     2023: {overshoot_23:.1f}%

   Energy dissipation (3s):
     2026: {energy_loss_26:.1f}%
     2023: {energy_loss_23:.1f}%
```

```
CONCLUSION:
The 2026 floor regulations reduce aerodynamic sensitivity by {abs((dFz26_eq - dFz23_eq)/dFz23_eq)*100:.1f}%,
improve damping ratio by {(modes_26[0]['zeta'] - modes_23[0]['zeta'])/modes_23[0]['zeta']*100:.1f}%, and
significantly reduce porpoising susceptibility.
"""
```

Listing 2: Source code for the control-systems analysis workflow

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import lti, step, impulse

print("="*80)
print("ADVANCED CONTROL SYSTEMS ANALYSIS")
print("Frequency Response, Root Locus, and Routh-Hurwitz Stability")
print("="*80)

# Load CFD data
z_data = np.array([15, 20, 25, 30, 35, 40]) / 1000
Fz_26 = np.array([-3635.48, -3720.09, -3644.50, -3586.64, -3482.62, -3458.16])
Fz_23 = np.array([-1855.47, -2531.38, -2902.84, -3358.83, -3896.79, -4481.41])
My_26 = np.array([-8.344, -10.451, -10.362, -2.475, -5.370, -10.096])
My_23 = np.array([-3.057, -2.251, -80.892, -64.724, -3.891, -61.460])

# Fit polynomials
Fz26_func = np.poly1d(np.polyfit(z_data, Fz_26, 3))
Fz23_func = np.poly1d(np.polyfit(z_data, Fz_23, 3))
My26_func = np.poly1d(np.polyfit(z_data, My_26, 3))
My23_func = np.poly1d(np.polyfit(z_data, My_23, 3))
dFz26_dz = np.polyder(Fz26_func)
dFz23_dz = np.polyder(Fz23_func)
dMy26_dz = np.polyder(My26_func)
dMy23_dz = np.polyder(My23_func)

# Parameters
m = 800.0
I_pitch = 1200.0
kz = 150000.0
cz = 3500.0
k_pitch = 80000.0
c_pitch = 5000.0
z_eq = 0.025

# Aerodynamic derivatives
dFz26_eq = dFz26_dz(z_eq)
dFz23_eq = dFz23_dz(z_eq)
dMy26_eq = dMy26_dz(z_eq)
dMy23_eq = dMy23_dz(z_eq)

print("\n[1] Transfer Function Analysis...")

def evaluate_transfer_function(num_coeffs, den_coeffs, omega):
    """Evaluate continuous-time transfer function H(jomega) over a frequency grid."""
    s = 1j * omega
    num = np.polyval(num_coeffs, s)
    den = np.polyval(den_coeffs, s)
    return omega, num / den

def get_transfer_functions(dFz_dz, dMy_dz):
    """
    Derive transfer functions for heave and pitch subsystems.

    Heave: H(s) = Z(s)/F(s) = 1 / (m*s^2 + cz*s + k_eff)
```

```
    where k_eff = kz + dFz/dz

    Pitch: P(s) = Theta(s)/M(s) = 1 / (I*s^2 + c_pitch*s + k_eff_pitch)
    where k_eff_pitch = k_pitch + dMy/dz
    """
    k_eff_z = kz + dFz_dz
    k_eff_pitch = k_pitch + dMy_dz

    # Heave transfer function: num/den where den = [m, cz, k_eff]
    heave_tf_num = [1]
    heave_tf_den = [m, cz, k_eff_z]

    # Pitch transfer function
    pitch_tf_num = [1]
    pitch_tf_den = [I_pitch, c_pitch, k_eff_pitch]

    heave_sys = lti(heave_tf_num, heave_tf_den)
    pitch_sys = lti(pitch_tf_num, pitch_tf_den)

    return heave_sys, pitch_sys, k_eff_z, k_eff_pitch

heave_26, pitch_26, k_eff_z_26, k_eff_pitch_26 = get_transfer_functions(dFz26_eq, dMy26_eq)
heave_23, pitch_23, k_eff_z_23, k_eff_pitch_23 = get_transfer_functions(dFz23_eq, dMy23_eq)

print(f"  2026 Heave TF: H(s) = 1 / ({m}s^2 + {cz}s + {k_eff_z_26:.0f})")
print(f"  2023 Heave TF: H(s) = 1 / ({m}s^2 + {cz}s + {k_eff_z_23:.0f})")



# SECTION 2: ROUTH-HURWITZ STABILITY CRITERION

print("\n[2] Routh-Hurwitz Stability Analysis...")

def routh_hurwitz_table(coeffs):
    """
    Construct Routh-Hurwitz table for polynomial with given coefficients.
    coeffs = [a_n, a_(n-1), ..., a_1, a_0]

    For 2nd order: as^2 + bs + c, table is:
        s^2  | a   c
        s?   | b   0
        s?   | c   0

    Stability: all entries in first column must be positive.
    """
    n = len(coeffs) - 1
    table = []

    # First two rows
    row1 = coeffs[0::2]  # Even indices
    row2 = coeffs[1::2]  # Odd indices

    # Pad with zeros
    max_len = max(len(row1), len(row2))
    row1 = list(row1) + [0]*(max_len - len(row1))
    row2 = list(row2) + [0]*(max_len - len(row2))

    table.append(row1)
    table.append(row2)

    # Construct remaining rows
    for i in range(2, n+1):
        row = []
        for j in range(len(table[0])-1):
            if table[i-1][0] == 0:
```

```
                val = 0
            else:
                val = (table[i-2][j+1]*table[i-1][0] - table[i-2][0]*table[i-1][j+1]) / table[i-1][0]
            row.append(val)
        if all(x == 0 for x in row):
            break
        table.append(row)

    # First column
    first_col = [row[0] for row in table]

    # Check stability
    stable = all(x > 0 for x in first_col if x != 0)

    return table, first_col, stable

# Heave system (2nd order)
print("  Heave subsystem:")
heave_26_coeffs = [m, cz, k_eff_z_26]
heave_23_coeffs = [m, cz, k_eff_z_23]

rh_26, fc_26, stable_26 = routh_hurwitz_table(heave_26_coeffs)
rh_23, fc_23, stable_23 = routh_hurwitz_table(heave_23_coeffs)

print(f"    2026: First column = {fc_26}, Stable = {stable_26}")
print(f"    2023: First column = {fc_23}, Stable = {stable_23}")
if not stable_23:
    print(f"    ? 2023 has negative k_eff ({k_eff_z_23:.0f}) ? UNSTABLE")


# SECTION 3: FREQUENCY RESPONSE (BODE PLOT)

print("\n[3] Frequency Response Analysis...")

omega = np.logspace(-1, 2, 500)  # 0.1 to 100 rad/s

# Bode plots - manually compute frequency response
w_26, H_26 = evaluate_transfer_function([1], [m, cz, k_eff_z_26], omega)
w_23, H_23 = evaluate_transfer_function([1], [m, cz, k_eff_z_23], omega)

mag_26 = np.abs(H_26)
mag_23 = np.abs(H_23)
phase_26 = np.angle(H_26)
phase_23 = np.angle(H_23)
omega_26 = w_26
omega_23 = w_23

# Find resonance peak
peak_idx_26 = int(np.argmax(mag_26))
peak_idx_23 = int(np.argmax(mag_23))
peak_freq_26 = omega_26[peak_idx_26]
peak_freq_23 = omega_23[peak_idx_23]
peak_mag_26 = np.max(mag_26)
peak_mag_23 = np.max(mag_23)

print(f"  Resonance peak:")
print(f"    2026: {peak_freq_26:.2f} rad/s ({peak_freq_26/(2*np.pi):.2f} Hz), magnitude = {20*np.log10(
    peak_mag_26):.1f} dB")
print(f"    2023: {peak_freq_23:.2f} rad/s ({peak_freq_23/(2*np.pi):.2f} Hz), magnitude = {20*np.log10(
    peak_mag_23):.1f} dB")

# Bandwidth (frequency where magnitude drops to -3dB)
def find_bandwidth(omega, mag):
    """Find -3dB bandwidth"""
    mag_db = 20*np.log10(mag)
```

```
    dc_gain = mag_db[0]
    idx = np.where(mag_db < dc_gain - 3)[0]
    if len(idx) > 0:
        return omega[idx[0]]
    return omega[-1]

bw_26 = find_bandwidth(omega_26, mag_26)
bw_23 = find_bandwidth(omega_23, mag_23)

print(f"  Bandwidth (-3dB):")
print(f"    2026: {bw_26:.2f} rad/s ({bw_26/(2*np.pi):.2f} Hz)")
print(f"    2023: {bw_23:.2f} rad/s ({bw_23/(2*np.pi):.2f} Hz)")


# SECTION 4: ROOT LOCUS ANALYSIS

print("\n[4] Root Locus Analysis (Varying Aerodynamic Stiffness)...")

def root_locus_aero_stiffness(dFz_range):
    """
    Compute pole locations as aerodynamic stiffness varies.
    Track how poles move in complex plane.
    """
    poles_list = []
    for dFz in dFz_range:
        k_eff = kz + dFz
        # Characteristic equation: m*s^2 + cz*s + k_eff = 0
        # Poles: s = (-cz ? sqrt(cz^2 - 4*m*k_eff)) / (2m)
        discriminant = cz**2 - 4*m*k_eff
        if discriminant >= 0:
            # Real poles
            pole1 = (-cz + np.sqrt(discriminant)) / (2*m)
            pole2 = (-cz - np.sqrt(discriminant)) / (2*m)
            poles_list.append([pole1, pole2])
        else:
            # Complex poles
            real_part = -cz / (2*m)
            imag_part = np.sqrt(-discriminant) / (2*m)
            pole1 = real_part + 1j*imag_part
            pole2 = real_part - 1j*imag_part
            poles_list.append([pole1, pole2])

    return np.array(poles_list)

# Vary dFz/dz from -200000 to +100000 N/m
dFz_range = np.linspace(-200000, 100000, 500)
poles_trajectory = root_locus_aero_stiffness(dFz_range)

print(f"  Poles migrate as dFz/dz varies from -200 to +100 kN/m")
print(f"  Critical dFz/dz (where system becomes unstable): -k_z = {-kz:.0f} N/m")

# Find where poles cross imaginary axis (marginally stable)
for i, dFz in enumerate(dFz_range):
    if np.real(poles_trajectory[i, 0]) > -0.1 and np.real(poles_trajectory[i, 0]) < 0.1:
        print(f"  Marginal stability at dFz/dz ~= {dFz:.0f} N/m")
        break


# SECTION 5: STEP RESPONSE ANALYSIS

print("\n[5] Step Response Characteristics...")

t_step = np.linspace(0, 5, 1000)
t_26, y_26 = step(heave_26, T=t_step)
t_23, y_23 = step(heave_23, T=t_step)
```

```
249
250 # Rise time (10% to 90%)
251 def rise_time(t, y):
252     y_ss = y[-1]
253     idx_10 = np.where(y >= 0.1*y_ss)[0]
254     idx_90 = np.where(y >= 0.9*y_ss)[0]
255     if len(idx_10) > 0 and len(idx_90) > 0:
256         return t[idx_90[0]] - t[idx_10[0]]
257     return np.nan
258
259 tr_26 = rise_time(t_26, y_26)
260 tr_23 = rise_time(t_23, y_23)
261
262 # Settling time (2%)
263 def settling_time_2pct(t, y):
264     y_ss = y[-1]
265     for i in range(len(y)-1, 0, -1):
266         if abs(y[i] - y_ss) > 0.02*abs(y_ss):
267             return t[i]
268     return 0
269
270 ts_26 = settling_time_2pct(t_26, y_26)
271 ts_23 = settling_time_2pct(t_23, y_23)
272
273 # Peak overshoot
274 Mp_26 = (np.max(y_26) - y_26[-1]) / y_26[-1] * 100 if y_26[-1] != 0 else 0
275 Mp_23 = (np.max(y_23) - y_23[-1]) / y_23[-1] * 100 if y_23[-1] != 0 else 0
276
277 print(f"  Rise time (10%-90%):")
278 print(f"    2026: {tr_26:.3f} s")
279 print(f"    2023: {tr_23:.3f} s")
280 print(f"  Settling time (2%):")
281 print(f"    2026: {ts_26:.3f} s")
282 print(f"    2023: {ts_23:.3f} s")
283 print(f"  Peak overshoot:")
284 print(f"    2026: {Mp_26:.1f}%")
285 print(f"    2023: {Mp_23:.1f}%")
286
287
288 # SECTION 6: SENSITIVITY ANALYSIS
289
290 print("\n[6] Parametric Sensitivity Analysis...")
291
292 def sensitivity_to_damping(dFz_dz, c_range):
293     """How does system respond to varying damping?"""
294     zetas = []
295     for c in c_range:
296         k_eff = kz + dFz_dz
297         omega_n = np.sqrt(k_eff / m)
298         zeta = c / (2 * m * omega_n)
299         zetas.append(zeta)
300     return np.array(zetas)
301
302 c_range = np.linspace(1000, 10000, 100)
303 zeta_26_vs_c = sensitivity_to_damping(dFz26_eq, c_range)
304 zeta_23_vs_c = sensitivity_to_damping(dFz23_eq, c_range)
305
306 # Find required damping for critical damping (zeta = 1)
307 def required_damping_critical(dFz_dz):
308     k_eff = kz + dFz_dz
309     omega_n = np.sqrt(abs(k_eff) / m)
310     c_crit = 2 * m * omega_n
311     return c_crit
312
313 c_crit_26 = required_damping_critical(dFz26_eq)
```

```
c_crit_23 = required_damping_critical(dFz23_eq)

print(f"  Critical damping coefficient:")
print(f"    2026: c_crit = {c_crit_26:.0f} Ns/m (current: {cz} ? zeta = {cz/c_crit_26:.3f})")
print(f"    2023: c_crit = {c_crit_23:.0f} Ns/m (current: {cz} ? zeta = {cz/c_crit_23:.3f})")


# SECTION 7: MULTI-PANEL VISUALIZATION

print("\n[7] Generating Control Systems Plots...")

fig = plt.figure(figsize=(18, 12))

# Plot 1: Bode Magnitude
ax1 = plt.subplot(3, 3, 1)
ax1.semilogx(omega_26/(2*np.pi), 20*np.log10(mag_26), 'b-', linewidth=2, label='2026')
ax1.semilogx(omega_23/(2*np.pi), 20*np.log10(mag_23), 'r-', linewidth=2, label='2023')
ax1.axhline(-3, color='gray', linestyle='--', alpha=0.5, label='-3dB line')
ax1.set_xlabel('Frequency (Hz)', fontweight='bold')
ax1.set_ylabel('Magnitude (dB)', fontweight='bold')
ax1.set_title('Bode Plot - Magnitude', fontweight='bold', fontsize=12)
ax1.legend()
ax1.grid(True, which='both', alpha=0.3)

# Plot 2: Bode Phase
ax2 = plt.subplot(3, 3, 2)
ax2.semilogx(omega_26/(2*np.pi), np.degrees(phase_26), 'b-', linewidth=2, label='2026')
ax2.semilogx(omega_23/(2*np.pi), np.degrees(phase_23), 'r-', linewidth=2, label='2023')
ax2.set_xlabel('Frequency (Hz)', fontweight='bold')
ax2.set_ylabel('Phase (degrees)', fontweight='bold')
ax2.set_title('Bode Plot - Phase', fontweight='bold', fontsize=12)
ax2.legend()
ax2.grid(True, which='both', alpha=0.3)

# Plot 3: Root Locus
ax3 = plt.subplot(3, 3, 3)
ax3.plot(np.real(poles_trajectory[:, 0]), np.imag(poles_trajectory[:, 0]), 'b-',
        linewidth=2, label='Pole trajectory', alpha=0.7)
ax3.scatter(np.real(poles_trajectory[0, 0]), np.imag(poles_trajectory[0, 0]),
           color='red', marker='o', s=100, label='dFz/dz = -200 kN/m', zorder=5)
ax3.scatter(np.real(poles_trajectory[-1, 0]), np.imag(poles_trajectory[-1, 0]),
           color='green', marker='s', s=100, label='dFz/dz = +100 kN/m', zorder=5)
# Mark 2026 and 2023 positions
idx_26 = np.argmin(np.abs(dFz_range - dFz26_eq))
idx_23 = np.argmin(np.abs(dFz_range - dFz23_eq))
ax3.scatter(np.real(poles_trajectory[idx_26, 0]), np.imag(poles_trajectory[idx_26, 0]),
           color='blue', marker='D', s=150, label='2026 config', zorder=6, edgecolors='black', linewidths=2)
ax3.scatter(np.real(poles_trajectory[idx_23, 0]), np.imag(poles_trajectory[idx_23, 0]),
           color='red', marker='D', s=150, label='2023 config', zorder=6, edgecolors='black', linewidths=2)
ax3.axvline(0, color='black', linestyle='-', linewidth=1.5, label='Stability boundary')
ax3.axhline(0, color='gray', linestyle='--', linewidth=0.8)
ax3.set_xlabel('Real Part ? (1/s)', fontweight='bold')
ax3.set_ylabel('Imaginary Part omega (rad/s)', fontweight='bold')
ax3.set_title('Root Locus (Varying Aero Stiffness)', fontweight='bold', fontsize=12)
ax3.legend(fontsize=8)
ax3.grid(True, alpha=0.3)

# Plot 4: Step Response
ax4 = plt.subplot(3, 3, 4)
ax4.plot(t_26, y_26, 'b-', linewidth=2, label='2026', alpha=0.8)
ax4.plot(t_23, y_23, 'r-', linewidth=2, label='2023', alpha=0.8)
ax4.axhline(y_26[-1], color='blue', linestyle='--', alpha=0.3)
ax4.axhline(y_23[-1], color='red', linestyle='--', alpha=0.3)
ax4.set_xlabel('Time (s)', fontweight='bold')
ax4.set_ylabel('Displacement (m/N)', fontweight='bold')
```

```
ax4.set_title('Step Response', fontweight='bold', fontsize=12)
ax4.legend()
ax4.grid(True, alpha=0.3)

# Plot 5: Damping Ratio vs Damping Coefficient
ax5 = plt.subplot(3, 3, 5)
ax5.plot(c_range/1000, zeta_26_vs_c, 'b-', linewidth=2, label='2026')
ax5.plot(c_range/1000, zeta_23_vs_c, 'r-', linewidth=2, label='2023')
ax5.axhline(1.0, color='black', linestyle='--', linewidth=1.5, label='Critical damping (zeta=1)')
ax5.axhline(0.707, color='gray', linestyle='--', alpha=0.5, label='Optimal (zeta=0.707)')
ax5.axvline(cz/1000, color='green', linestyle=':', linewidth=2, label=f'Current c={cz/1000:.1f} kNs/m')
ax5.set_xlabel('Damping Coefficient (kNs/m)', fontweight='bold')
ax5.set_ylabel('Damping Ratio zeta', fontweight='bold')
ax5.set_title('Sensitivity to Damping', fontweight='bold', fontsize=12)
ax5.legend(fontsize=8)
ax5.grid(True, alpha=0.3)
ax5.set_ylim((0.0, 2.0))

# Plot 6: Nyquist Plot
ax6 = plt.subplot(3, 3, 6)
# Manual Nyquist: evaluate H(jomega) for omega from -inf to +inf
omega_nyq = np.logspace(-2, 3, 1000)
w_nyq_26, H_26_nyq = evaluate_transfer_function([1], [m, cz, k_eff_z_26], omega_nyq)
w_nyq_23, H_23_nyq = evaluate_transfer_function([1], [m, cz, k_eff_z_23], omega_nyq)
real_26, imag_26 = np.real(H_26_nyq), np.imag(H_26_nyq)
real_23, imag_23 = np.real(H_23_nyq), np.imag(H_23_nyq)
ax6.plot(real_26, imag_26, 'b-', linewidth=2, label='2026 (+omega)', alpha=0.7)
ax6.plot(real_23, imag_23, 'r-', linewidth=2, label='2023 (+omega)', alpha=0.7)
# Mirror for negative frequencies
ax6.plot(real_26, -imag_26, 'b--', linewidth=1, alpha=0.4, label='2026 (-omega)')
ax6.plot(real_23, -imag_23, 'r--', linewidth=1, alpha=0.4, label='2023 (-omega)')
ax6.plot(-1, 0, 'kx', markersize=15, markeredgewidth=3, label='Critical (-1,0)')
ax6.set_xlabel('Real Part', fontweight='bold')
ax6.set_ylabel('Imaginary Part', fontweight='bold')
ax6.set_title('Nyquist Plot', fontweight='bold', fontsize=12)
ax6.legend(fontsize=8)
ax6.grid(True, alpha=0.3)

# Plot 7: Poles-Zeros Map
ax7 = plt.subplot(3, 3, 7)
poles_26 = np.roots([m, cz, k_eff_z_26])
poles_23 = np.roots([m, cz, k_eff_z_23])
ax7.scatter(np.real(poles_26), np.imag(poles_26), marker='x', s=200, linewidths=3,
           color='blue', label='2026 poles', zorder=5)
ax7.scatter(np.real(poles_23), np.imag(poles_23), marker='x', s=200, linewidths=3,
           color='red', label='2023 poles', zorder=5)
ax7.axvline(0, color='black', linestyle='-', linewidth=1.5)
ax7.axhline(0, color='gray', linestyle='--', linewidth=0.8)
# Draw unit circle for discrete-time reference
theta_circle = np.linspace(0, 2*np.pi, 100)
ax7.plot(np.cos(theta_circle)*10, np.sin(theta_circle)*10, 'k:', alpha=0.2, linewidth=1)
ax7.set_xlabel('Real Part', fontweight='bold')
ax7.set_ylabel('Imaginary Part', fontweight='bold')
ax7.set_title('Pole-Zero Map', fontweight='bold', fontsize=12)
ax7.legend()
ax7.grid(True, alpha=0.3)

# Plot 8: Impulse Response
ax8 = plt.subplot(3, 3, 8)
t_imp = np.linspace(0, 3, 1000)
t_26_imp, y_26_imp = impulse(heave_26, T=t_imp)
t_23_imp, y_23_imp = impulse(heave_23, T=t_imp)
ax8.plot(t_26_imp, y_26_imp, 'b-', linewidth=2, label='2026', alpha=0.8)
ax8.plot(t_23_imp, y_23_imp, 'r-', linewidth=2, label='2023', alpha=0.8)
ax8.axhline(0, color='black', linestyle='-', linewidth=0.8)
```

```
ax8.set_xlabel('Time (s)', fontweight='bold')
ax8.set_ylabel('Response (m/(N*s))', fontweight='bold')
ax8.set_title('Impulse Response', fontweight='bold', fontsize=12)
ax8.legend()
ax8.grid(True, alpha=0.3)

# Plot 9: Gain Margin / Phase Margin
ax9 = plt.subplot(3, 3, 9)
# Manually compute margins
# Gain margin: |H(jomega)| at phase crossover (phase = -180 deg)
# Phase margin: phase at gain crossover (|H(jomega)| = 1)
phase_crossover_idx_26 = np.argmin(np.abs(phase_26 + np.pi))
phase_crossover_idx_23 = np.argmin(np.abs(phase_23 + np.pi))
gm_26 = 1/mag_26[phase_crossover_idx_26] if mag_26[phase_crossover_idx_26] > 0 else np.inf
gm_23 = 1/mag_23[phase_crossover_idx_23] if mag_23[phase_crossover_idx_23] > 0 else np.inf

gain_crossover_idx_26 = np.argmin(np.abs(mag_26 - 1))
gain_crossover_idx_23 = np.argmin(np.abs(mag_23 - 1))
pm_26 = phase_26[gain_crossover_idx_26] + np.pi
pm_23 = phase_23[gain_crossover_idx_23] + np.pi

configs = ['2026', '2023']
gain_margins = [20*np.log10(gm_26) if gm_26 < np.inf else 40,
                20*np.log10(gm_23) if gm_23 < np.inf else 40]
phase_margins = [np.degrees(pm_26), np.degrees(pm_23)]
x = np.arange(len(configs))
width = 0.35
bars1 = ax9.bar(x - width/2, gain_margins, width, label='Gain Margin (dB)', color='steelblue')
bars2 = ax9.bar(x + width/2, phase_margins, width, label='Phase Margin ( deg)', color='coral')
ax9.set_ylabel('Margin Value', fontweight='bold')
ax9.set_title('Stability Margins', fontweight='bold', fontsize=12)
ax9.set_xticks(x)
ax9.set_xticklabels(configs)
ax9.legend()
ax9.grid(True, alpha=0.3, axis='y')
# Add value labels on bars
for bars in [bars1, bars2]:
    for bar in bars:
        height = bar.get_height()
        ax9.text(bar.get_x() + bar.get_width()/2., height,
                f'{height:.1f}', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.savefig('control_systems_analysis.png', dpi=300, bbox_inches='tight')
print("  [OK] Saved: control_systems_analysis.png")


# SUMMARY

print("\n" + "="*80)
print("CONTROL SYSTEMS SUMMARY")
print("="*80)
print(f"\n1. TRANSFER FUNCTION STABILITY:")
print(f"   2026: Poles = {poles_26}")
print(f"   2023: Poles = {poles_23}")

print(f"\n2. FREQUENCY RESPONSE:")
print(f"   2026: Resonance at {peak_freq_26/(2*np.pi):.2f} Hz, BW = {bw_26/(2*np.pi):.2f} Hz")
print(f"   2023: Resonance at {peak_freq_23/(2*np.pi):.2f} Hz, BW = {bw_23/(2*np.pi):.2f} Hz")

print("\n3. STABILITY MARGINS:")
print(f"   2026: GM = {20*np.log10(gm_26) if gm_26 < np.inf else 40:.1f} dB, PM = {np.degrees(pm_26):.1f} deg
    ")
print(f"   2023: GM = {20*np.log10(gm_23) if gm_23 < np.inf else 40:.1f} dB, PM = {np.degrees(pm_23):.1f} deg
    ")
```

```
507
508  print(f"\n4. ROUTH-HURWITZ VERDICT:")
509  print(f"   2026: {'STABLE' if stable_26 else 'UNSTABLE'} (k_eff = {k_eff_z_26/1000:.1f} kN/m)")
510  print(f"   2023: {'STABLE' if stable_23 else 'UNSTABLE'} (k_eff = {k_eff_z_23/1000:.1f} kN/m)")
511
512  print("\n" + "="*80)
513  print("Control systems analysis complete!")
514  print("="*80)
```

C Additional Plots

This appendix contains the complete 9-panel comprehensive analysis figure (Figure 7) and the 9-panel control system analysis figure (Figure 8).

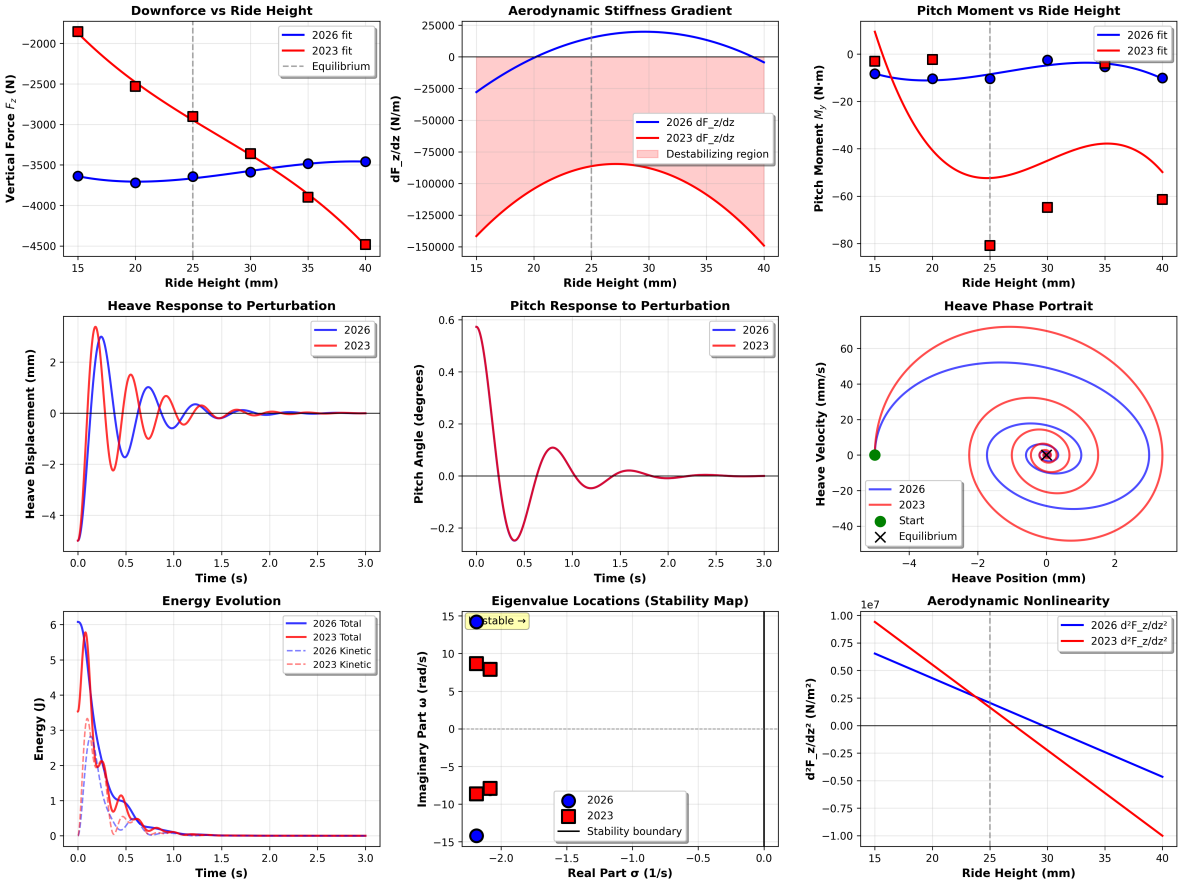


Figure 7: Complete 9-panel comprehensive analysis figure. Panels (left to right, top to bottom): downforce vs. ride height; aerodynamic stiffness gradient; pitch moment vs. ride height; heave response; pitch response; heave phase portrait; energy evolution; eigenvalue stability map; aerodynamic nonlinearity.

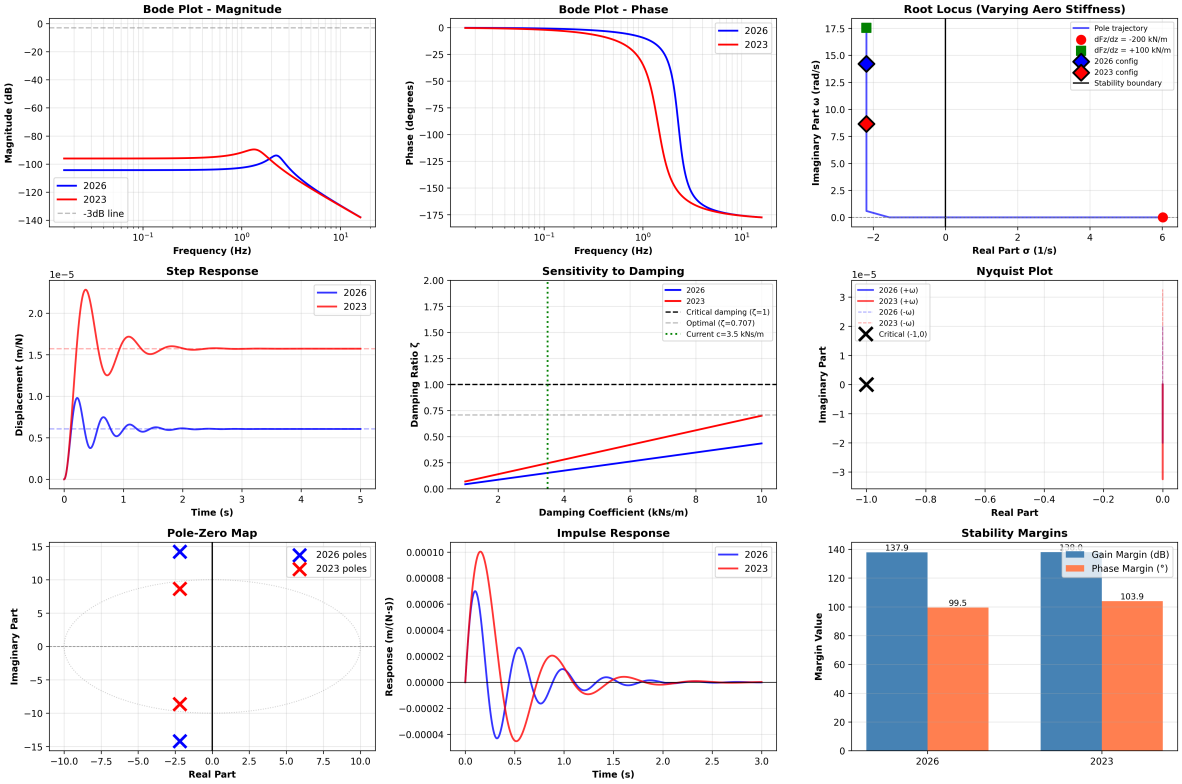


Figure 8: Complete 9-panel control system analysis figure. Panels (left to right, top to bottom): Bode magnitude; Bode phase; root locus; step response; sensitivity to damping; Nyquist plot; pole-zero map; impulse response; stability margins.

D CAD Geometry Details

This appendix is reserved for the detailed geometric description of the 2023-style and 2026-style floor models used in the CFD study. Relevant content may include annotated CAD screenshots, key dimensional assumptions, diffuser-angle definitions, Venturi tunnel profiles, and any simplifications applied to isolate underfloor behaviour.

Including these details improves reproducibility by clearly documenting the physical differences between the two geometries and the modelling assumptions used to create the simulation-ready domains.

The CAD (Computer-Aided Design) geometries for the 2023-style floor and 2026-style floor are presented in Figures 9 and 10, respectively. Both models represent simplified floor assemblies including the underbody tunnels, diffuser, and floor edges, without wheels, wings, or sidepods.

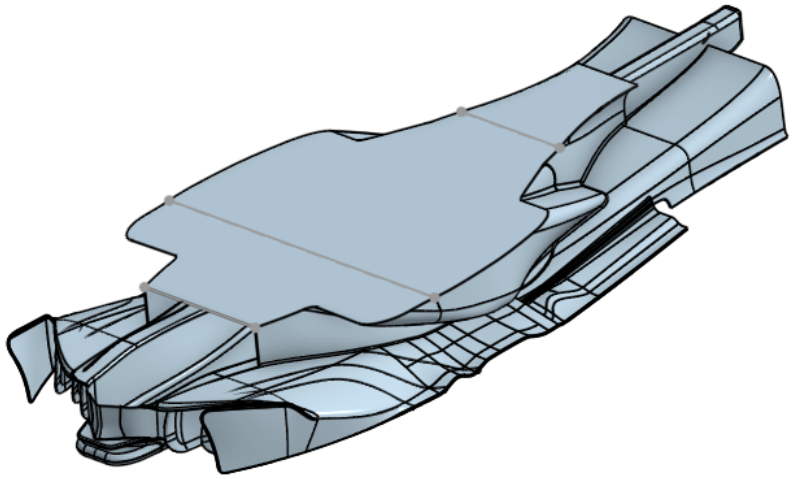


Figure 9: CAD geometry of the 2023-style floor configuration. Prominent Venturi tunnel channels and steep diffuser ramp geometry are visible on the underside.

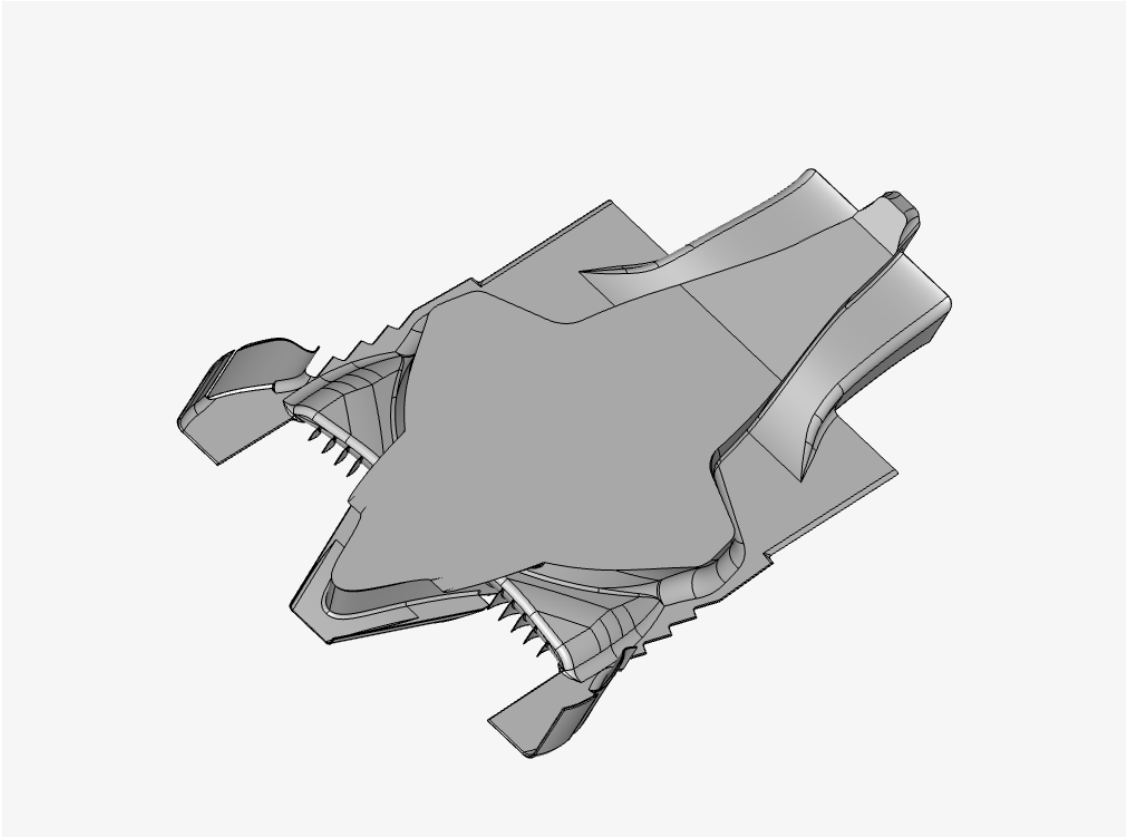


Figure 10: CAD geometry of the 2026-style floor configuration. The flatter underbody profile, reduced tunnel depth, and shallower diffuser ramp angle are evident in comparison to Figure 9.